Java & Spring Boot Internals

A Deep-Dive Reference: Concepts, Definitions & Hands-On Examples

---

1. Spring Boot Auto Configuration

2. @Bean vs @Component — Registering Third-Party Classes

3. Role-Based Access Control (RBAC) in Spring Security

4. Idempotency — Preventing Duplicate Payments

5. Java Class Loading Architecture

6. JFrog Artifactory

7. Functional Interfaces in Java

8. Dependency Injection (DI) in Spring

9. Observability in Spring Boot & Spring AI

10. CAP Theorem

11. Spring Security — Internal Filter Chain

12. Hibernate Entity Lifecycle

Part 2 — Hands-On Microservice Solutions

13. Solution: Third-Party Class as a Spring Bean

14. Solution: RBAC for a Banking Microservice

15. Solution: Preventing Duplicate Payments (Idempotency)

16. System Architecture Diagrams (Image 9) + Spring Boot Microservices Example

17. Functional Interfaces — Extended Hands-On

18. Dependency Injection — Extended Hands-On

19. Observability — Extended Hands-On

1. Spring Boot Auto Configuration

Definition

Auto Configuration is Spring Boot's mechanism for automatically wiring up beans and settings for your application based on what is present on the classpath, what beans already exist in the context, and what properties are defined in **application.properties** / **application.yml**. It removes the need for manual XML or Java configuration for common scenarios (embedded servers, data sources, JSON mapping, etc.).

How it Works Internally (Step by Step)

- **SpringApplication.run()** — the application boots from here.
- **Create ApplicationContext** — the IoC container is instantiated.
- **Component Scan** — Spring scans the project for **@Component, @Service, @Repository, @Controller**.
- **Classpath Check** — Spring inspects which libraries/dependencies are actually available.
- **Apply Auto Configuration** — conditional annotations decide what to configure based on the classpath.
- **Create & Register Beans** — required beans are instantiated and added to the context.
- **Start Embedded Server** — Tomcat (or Jetty/Undertow) starts by default.
- **Application Ready** — the app can now handle HTTP requests.

Key Annotations ("Key Players")

- **@EnableAutoConfiguration** — turns on the auto-configuration mechanism (bundled inside @SpringBootApplication).
- **@ConditionalOnClass** — applies configuration only if a given class is on the classpath.
- **@ConditionalOnMissingBean** — applies configuration only if the user hasn't already defined that bean.
- **@ConditionalOnProperty** — applies configuration only if a specific property is set to a given value.

Hands-On Example

A minimal custom auto-configuration class shows the pattern used internally by Spring Boot starters:

```
@Configuration
@ConditionalOnClass(DataSource.class)          // only if a DB driver is present
public class MyDataSourceAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean                  // don't override a user-defined bean
    public DataSource dataSource(
            @Value("${db.url}") String url) {
        return DataSourceBuilder.create()
                .url(url)
                .build();
    }
}
```

Why it matters: Auto configuration lets you add a dependency (e.g. spring-boot-starter-data-jpa) and get a fully working DataSource, EntityManager, and transaction manager without writing a line of config — while still letting you override anything by defining your own bean.



Fig 1. Spring Boot Auto Configuration — end-to-end flow from SpringApplication.run() to a ready application.

2. @Bean vs @Component — Registering Third-Party Classes

The Scenario

You have a class from a third-party library (e.g. a payment gateway SDK client such as **RazorPayGatewayClient**) and you need it managed as a Spring Bean. You cannot annotate the class itself with **@Component** because you don't own the source code.

Definition

- **@Component** (and its specializations @Service, @Repository, @Controller) is a class-level annotation. It requires you to have the source code so you can annotate the class, and Spring discovers it via component scanning.
- **@Bean** is a method-level annotation used inside a **@Configuration** class. It lets you manually construct and register any object as a bean, regardless of whether you own its source code.

The Rule of Thumb

- Own the class and it's part of your codebase → use **@Component**.
- Don't own the class (third-party library, JDK class, generated code) → use **@Bean** inside a @Configuration class.

Hands-On Example

```
// Third-party class — cannot be modified
public class RazorPayGatewayClient {
    private final String apiKey;
    public RazorPayGatewayClient(String apiKey) {
        this.apiKey = apiKey;
    }
    // 3rd-party library code...
}

// Your configuration class
@Configuration
public class PaymentConfig {

    @Bean
    public RazorPayGatewayClient razorPayGatewayClient(
            @Value("${razorpay.api.key}") String apiKey) {
        return new RazorPayGatewayClient(apiKey);
    }
}

// Usage anywhere in the app
@Service
public class PaymentService {
    private final RazorPayGatewayClient client;

    public PaymentService(RazorPayGatewayClient client) { // injected
        this.client = client;
    }
}
```

Why it matters: This pattern is the standard way to bring any external SDK, legacy class, or framework object under Spring's IoC container so it can be injected, mocked in tests, and lifecycle-managed like any other bean.

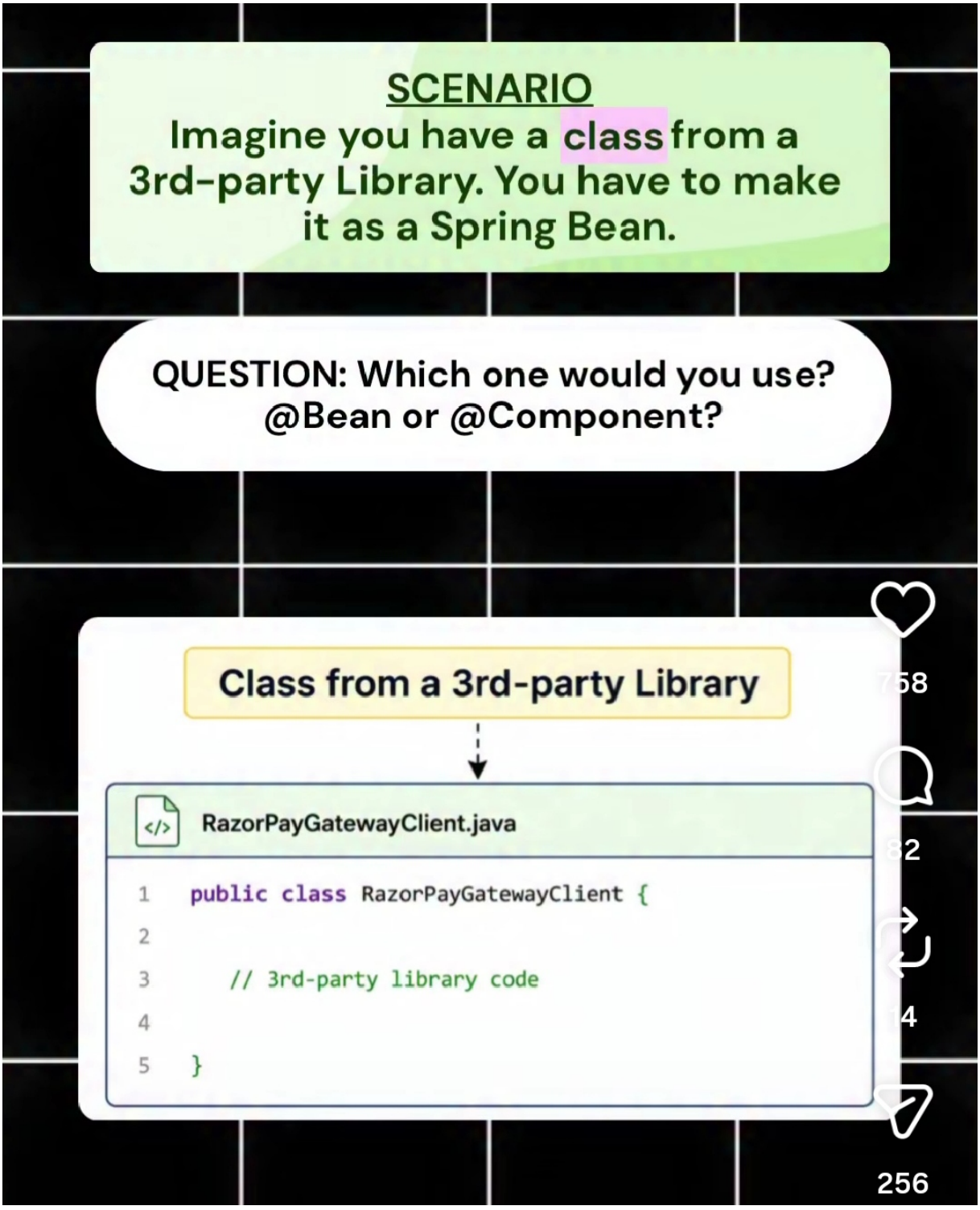


Fig 2. Wrapping a third-party class (RazorPayGatewayClient) as a Spring Bean using @Bean.

3. Role-Based Access Control (RBAC) in Spring Security

The Scenario

A banking application needs three permission tiers: **Consumers** can only view their own loan applications; **Employees** can create and view loan applications; **Managers** can create, view, and approve them.

Definition

Role-Based Access Control restricts what a user can do based on the **role/authority** assigned to their account. In Spring Security this is implemented with **GrantedAuthority** objects attached to the Authentication, then enforced with method-level or URL-level security rules.

Implementation Approach

- Assign roles to users: **ROLE_CONSUMER, ROLE_EMPLOYEE, ROLE_MANAGER**.
- Enable method security with **@EnableMethodSecurity**.
- Guard controller/service methods with **@PreAuthorize** using role expressions.
- Alternatively, restrict at the URL level using **HttpSecurity.authorizeHttpRequests()**.

Hands-On Example

```
@RestController
@RequestMapping("/loans")
public class LoanController {

    @GetMapping
    @PreAuthorize("hasAnyRole('CONSUMER','EMPLOYEE','MANAGER')")
    public List<Loan> viewLoans(Authentication auth) {
        return loanService.findVisibleTo(auth.getName());
    }

    @PostMapping
    @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
    public Loan createLoan(@RequestBody LoanRequest req) {
        return loanService.create(req);
    }

    @PostMapping("/{id}/approve")
    @PreAuthorize("hasRole('MANAGER')")
    public Loan approveLoan(@PathVariable Long id) {
        return loanService.approve(id);
    }
}

// SecurityConfig (URL-based alternative)
http.authorizeHttpRequests(auth -> auth
    .requestMatchers(HttpMethod.GET, "/loans/**").hasAnyRole("CONSUMER","EMPLOYEE","MANAGER")
    .requestMatchers(HttpMethod.POST, "/loans").hasAnyRole("EMPLOYEE","MANAGER")
    .requestMatchers(HttpMethod.POST, "/loans/*/approve").hasRole("MANAGER")
);
```

Why it matters: Layering @PreAuthorize on top of authentication ensures authorization decisions are declarative, testable, and centralized rather than scattered as if-checks throughout business logic.

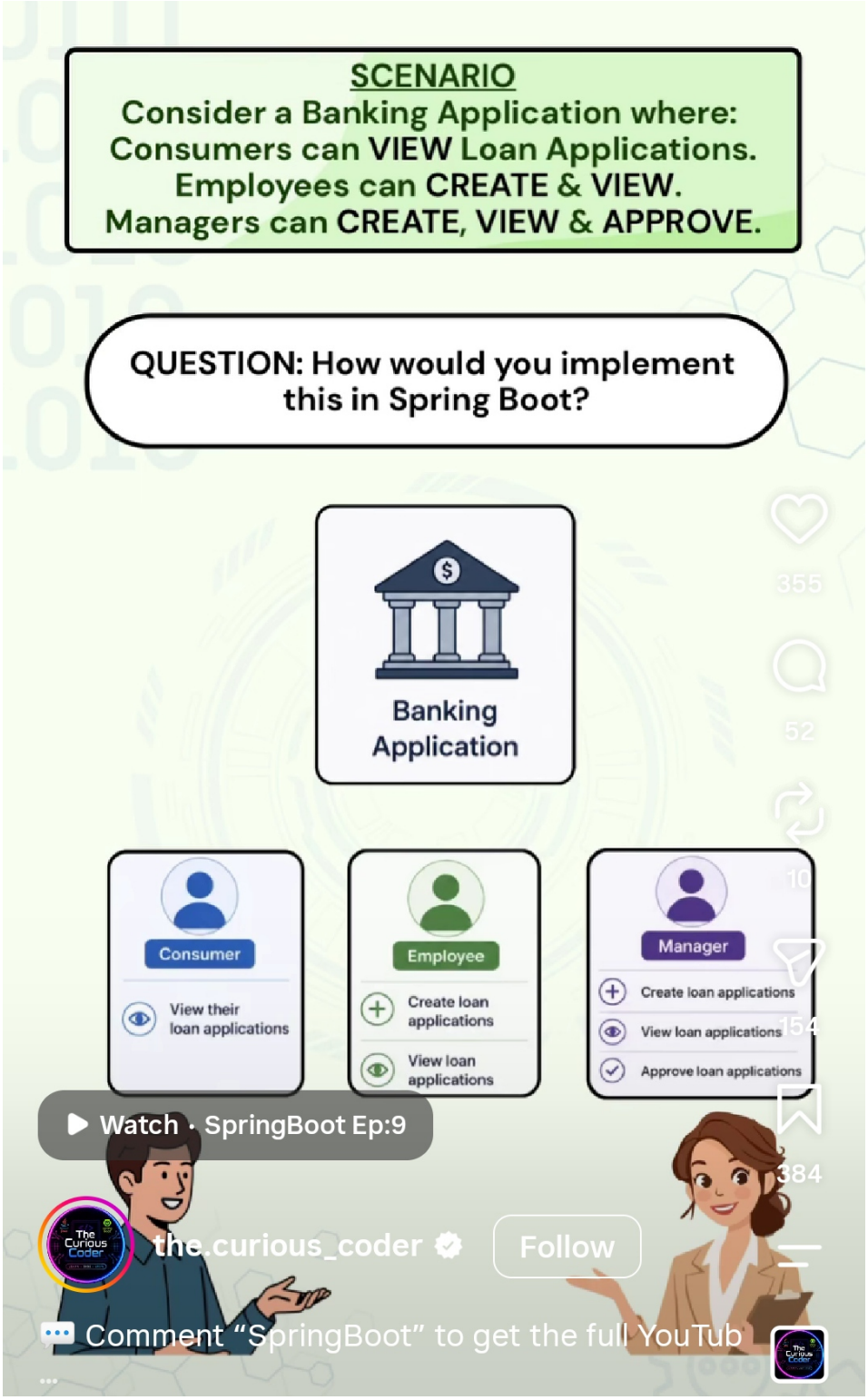


Fig 3. Role-based permission matrix for a banking application: Consumer, Employee, Manager.

4. Idempotency — Preventing Duplicate Payments

The Scenario

You're building a payment gateway. A customer clicks "Pay" twice because the network is slow. Without protection, this can trigger two separate charges for the same transaction.

Definition

Idempotency means that performing the same operation multiple times produces the same result as performing it once. An idempotent API guarantees that retried or duplicate requests do not cause duplicate side effects.

Implementation Strategy

- The client generates a unique **Idempotency-Key** (usually a UUID) before the first request attempt.
- The key is sent in a request header on every attempt (including retries).
- The server stores the key with the resulting response the first time it processes the request.
- On a repeated request with the same key, the server returns the **stored result** instead of re-executing the payment.
- A unique database constraint on the idempotency key acts as a safety net against race conditions.

Hands-On Example

```
@PostMapping("/payments")
public ResponseEntity<PaymentResponse> pay(
        @RequestHeader("Idempotency-Key") String idempotencyKey,
        @RequestBody PaymentRequest request) {

    Optional<PaymentRecord> existing = paymentRepo.findByIdempotencyKey(idempotencyKey);
    if (existing.isPresent()) {
        return ResponseEntity.ok(existing.get().toResponse()); // return cached result
    }

    try {
        PaymentRecord record = paymentService.process(request, idempotencyKey);
        return ResponseEntity.ok(record.toResponse());
    } catch (DataIntegrityViolationException e) {
        // Unique constraint on idempotency_key caught a race condition
        PaymentRecord record = paymentRepo.findByIdempotencyKey(idempotencyKey).orElseThrow();
        return ResponseEntity.ok(record.toResponse());
    }
}

// Table definition
// CREATE TABLE payments (
//   id BIGINT PRIMARY KEY,
//   idempotency_key VARCHAR(64) UNIQUE NOT NULL,
//   amount DECIMAL, status VARCHAR(20)
// );
```

Why it matters: Idempotency keys are the industry-standard solution used by Stripe, RazorPay, and most payment gateways to make retries safe over unreliable networks.

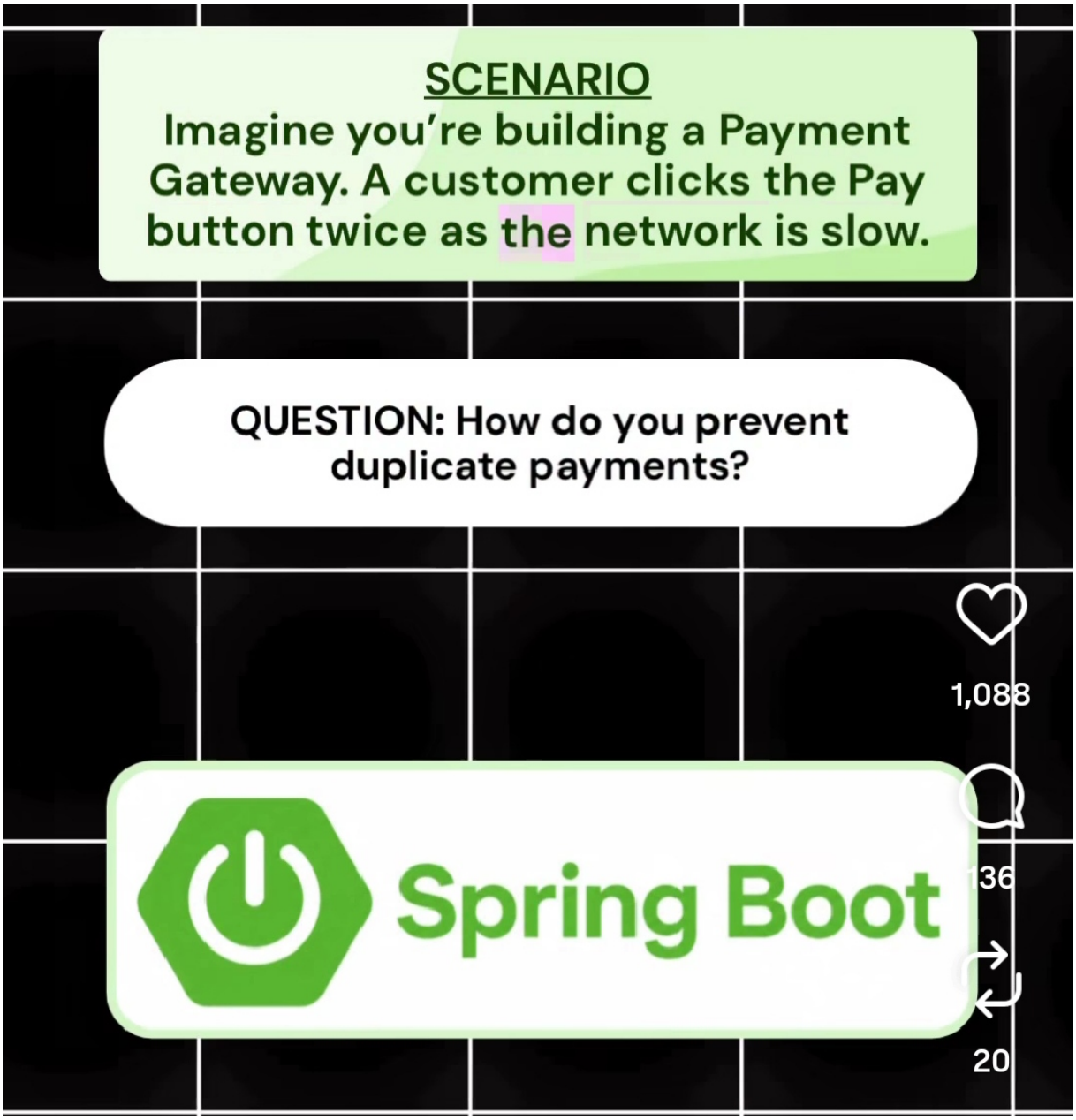


Fig 4. Double-click / slow-network scenario that idempotency keys are designed to solve.

5. Java Class Loading Architecture

Definition

The JVM does not load every class inside a JAR at startup. Instead, the **ClassLoader** follows a lazy-loading model: it loads a class into memory (the **Metaspace**) only the first time that class is actually referenced/needed during execution. Everything else stays untouched on disk inside the JAR.

Step-by-Step Flow

- **java -jar application.jar** starts the JVM; the ClassLoader looks for classes inside the JAR.
- For each class reference, the JVM asks: **"Is this class needed right now?"**
- **YES** → the class is loaded from the JAR into the **Metaspace** (part of JVM memory).
- **NO** → the class simply remains untouched inside the JAR on disk.
- Only the classes that were actually loaded live in Metaspace; a 200,000-class JAR might only load a few hundred.
- The JVM then executes the program using the classes that were loaded, via the **Stack** (method calls) and **Heap** (objects).

Why This Design is Fast & Efficient

- **Faster Startup** — only required classes are loaded, not the entire application.
- **Lower RAM Usage** — no need to hold all classes in memory.
- **Less CPU Overhead** — no unnecessary class-loading work.
- **Better Scalability** — large applications with huge dependency trees still start quickly.

Hands-On: Observing Lazy Loading

```
// Add -verbose:class to see exactly which classes get loaded and when
// java -verbose:class -jar application.jar

public class Demo {
    public static void main(String[] args) {
        System.out.println("Before referencing Helper class");
        // Helper.class is NOT loaded yet at this point

        Helper h = new Helper();     // <-- triggers class loading NOW
        h.doWork();

        System.out.println("After using Helper");
    }
}
// Output with -verbose:class shows [Loaded Helper from file:...]
// appearing right at 'new Helper()', not at JVM startup.
```

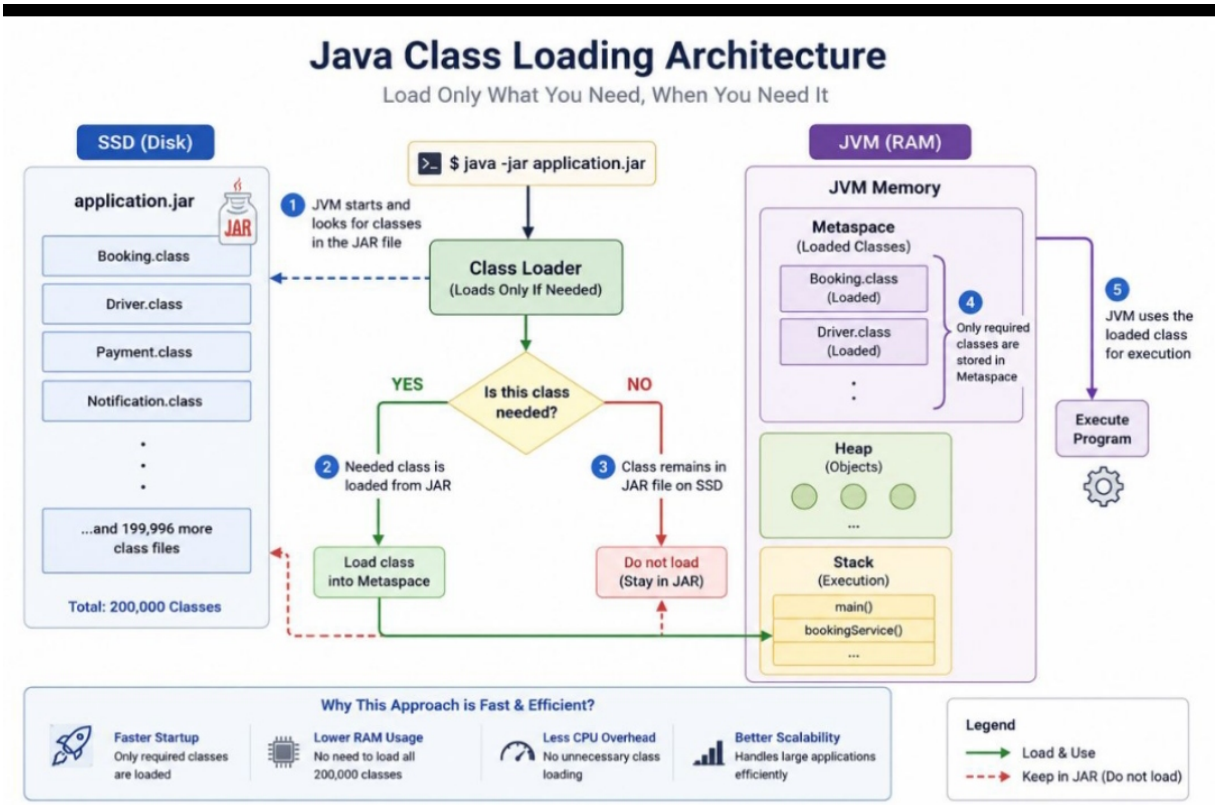


Fig 5. Java Class Loading Architecture — classes load into Metaspace only when needed.

6. JFrog Artifactory

Definition

JFrog Artifactory is a binary/artifact repository manager — a central storage place where companies store and share software packages (JARs, npm packages, Docker images, etc.). Think of it as "Google Drive for software packages," but with versioning, security scanning, and CI/CD integration built in.

The Problem Without Artifactory

Imagine a company with 500 developers. If everyone pulls the same third-party dependency directly from the public internet, you get:

- **Slow builds** — every machine re-downloads the same package from the internet.
- **Internet dependency** — builds fail if the public repository (e.g. Maven Central) is down or blocked.
- **Waste** — the same package gets downloaded hundreds of times across the org.

The Solution With Artifactory

Artifactory downloads the package **once**, caches it internally, and every developer/build server afterwards fetches it from the internal Artifactory instance instead of the public internet — faster, more reliable, and centrally controlled.

Hands-On: Pointing Maven/Gradle to Artifactory

```
<!-- Maven settings.xml -->
<repositories>
  <repository>
    <id>central-proxy</id>
    <url>https://mycompany.jfrog.io/artifactory/libs-release</url>
  </repository>
</repositories>

// Gradle build.gradle
repositories {
    maven {
        url "https://mycompany.jfrog.io/artifactory/libs-release"
        credentials {
            username = project.findProperty("artifactoryUser")
            password = project.findProperty("artifactoryPass")
        }
    }
}
```

Why it matters: Artifactory also enables governance — vulnerability scanning (Xray), promotion pipelines (dev → staging → release repos), and reproducible builds since artifacts are immutable once published.

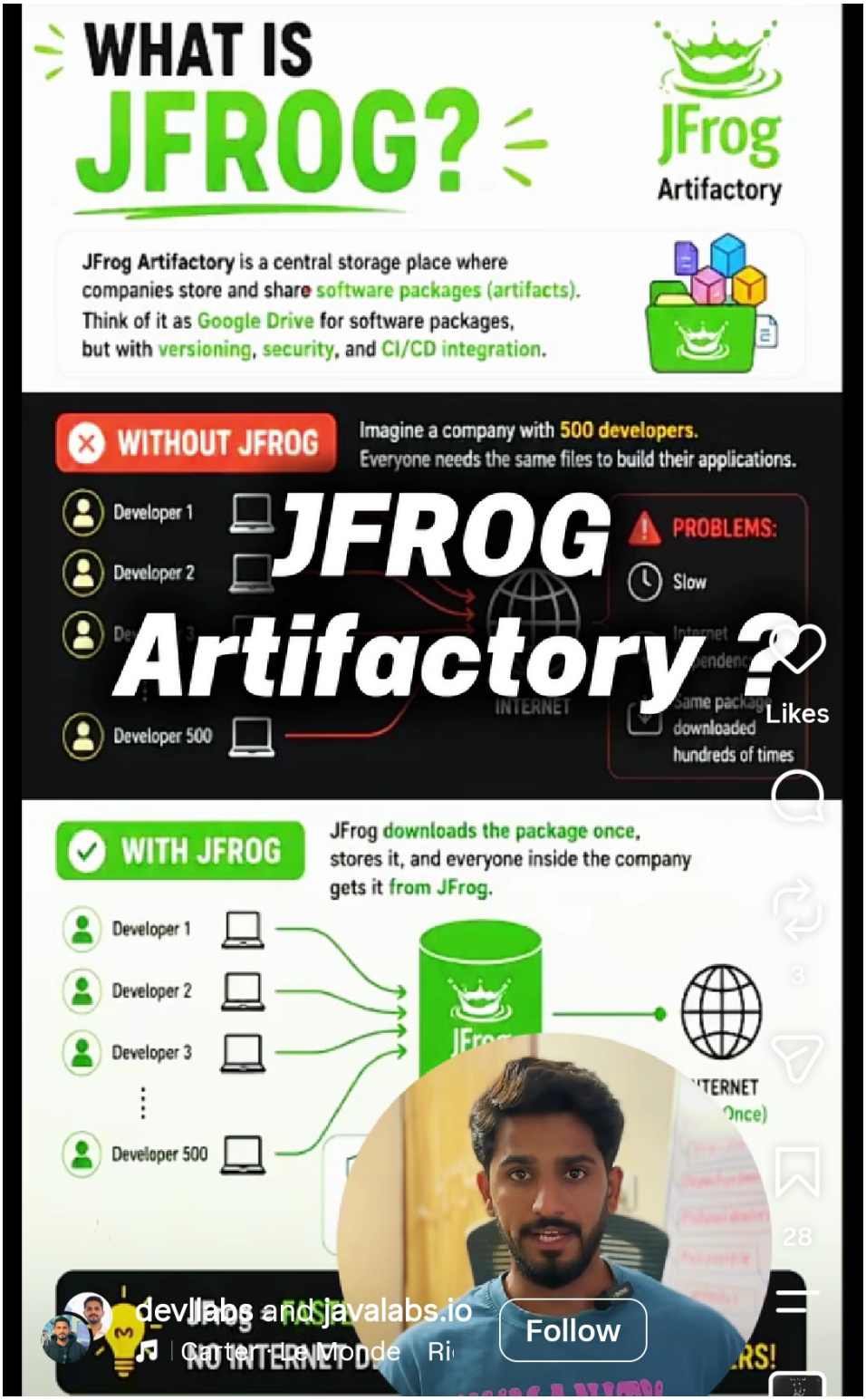


Fig 6. JFrog Artifactory — centralizing dependency downloads for an entire engineering org.

7. Functional Interfaces in Java

Definition

A **Functional Interface** is an interface that contains exactly **one abstract method** (Single Abstract Method / SAM). It may have any number of **default** or **static** methods. Functional interfaces are the foundation that makes Java **lambda expressions** possible.

Functional Interface vs Regular Interface

Aspect	Regular Interface	Functional Interface
Abstract Methods	0 to many	Exactly one
Lambda Support	No	Yes
Purpose	General contract	Functional programming
Annotation	None required	@FunctionalInterface (recommended)
Examples	List, Set, Map	Runnable, Comparator, Function, Predicate

Common Built-in Functional Interfaces (java.util.function)

- **Predicate<T>** — boolean test(T t): represents a condition (e.g. filter by age).
- **Function<T,R>** — R apply(T t): transforms input to output.
- **Consumer<T>** — void accept(T t): performs an action, returns nothing (e.g. print/save).
- **Supplier<T>** — T get(): takes no input, returns a value (e.g. generate/fetch data).
- **BiFunction<T,U,R>** — R apply(T t, U u): takes two inputs, returns a result.
- **UnaryOperator<T>** — T apply(T t): special case of Function where input and output types match.

Hands-On Example

```
@FunctionalInterface
interface MyPrinter {
    void print(String msg);   // exactly one abstract method
}

public class Demo {
    public static void main(String[] args) {
        // Lambda expression assigned to a functional interface
        MyPrinter printer = (msg) -> System.out.println(msg);
        printer.print("Hello, Functional Interface!");

        // Real-world use: filtering with Predicate
        Predicate<Employee> isSenior = e -> e.getAge() > 40;
        List<Employee> seniors = employees.stream()
                                          .filter(isSenior)
                                          .collect(Collectors.toList());
    }
}

// Invalid functional interface (compile error)
interface InvalidInterface {
    void m1();
    void m2();   // more than one abstract method -> NOT functional
}
```

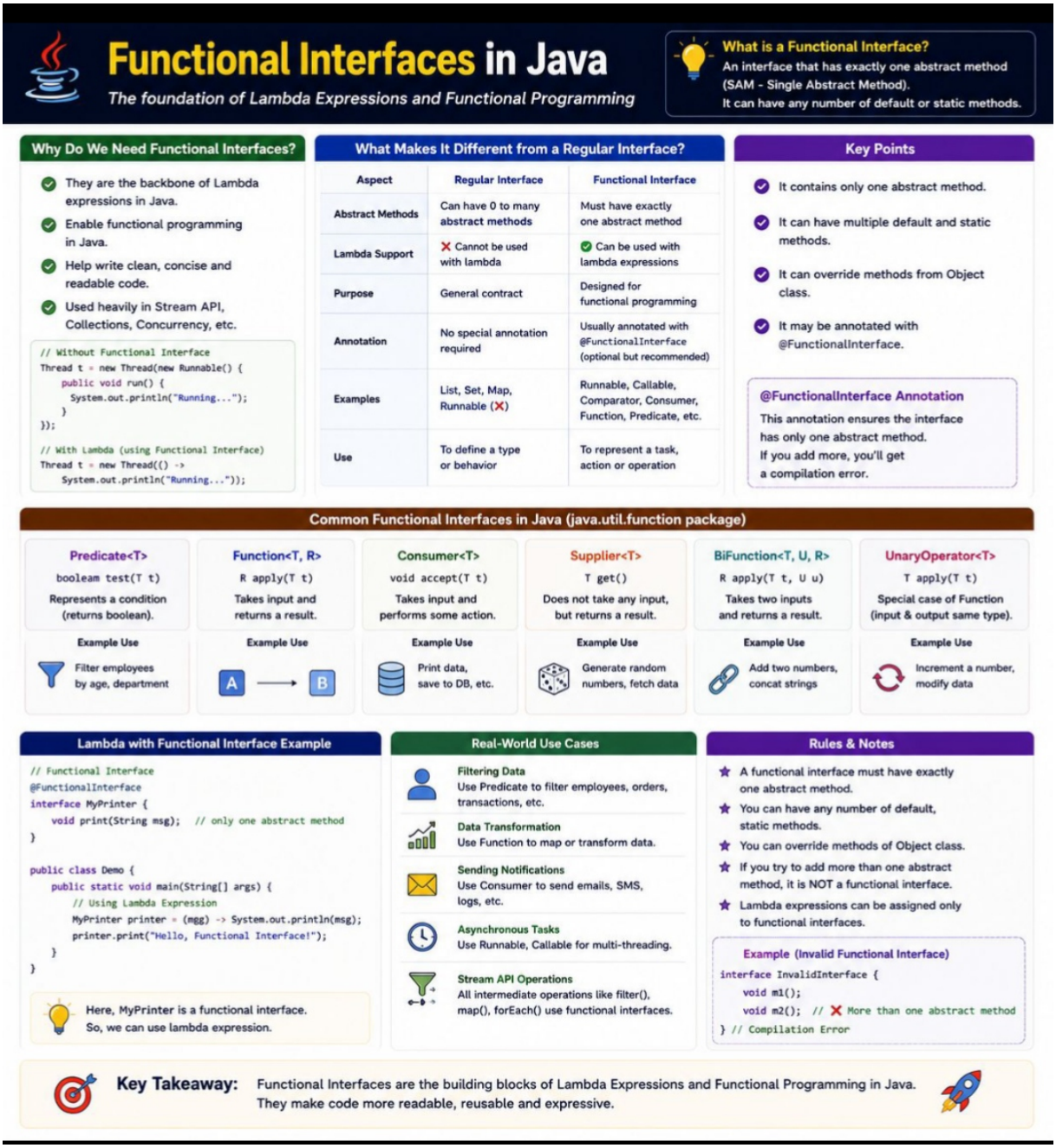


Fig 7. Functional Interfaces — the foundation of lambdas and the Stream API.

8. Dependency Injection (DI) in Spring

Definition

Dependency Injection is a design pattern where an object receives its dependencies from an external source (the Spring IoC Container) instead of creating them itself. In simple words: instead of your class creating what it needs with **new**, Spring provides it for you.

Real-World Analogy

Think of a restaurant: you (the customer) place an order — you don't go into the kitchen and cook it yourself. The chef (the Spring Container) prepares it and hands it to you. You depend on the restaurant; your class depends on Spring.

How It Works Internally

- Spring creates all objects (beans) and manages them inside the IoC Container.
- When Spring creates a bean, it inspects the bean's constructor/fields for required dependencies.
- Spring finds a matching bean already registered in the container.
- Spring injects that dependency into the requiring bean.
- The bean is now fully constructed and ready to use — all wiring is done automatically.

Hands-On Example — Constructor Injection (Recommended)

```
// BEFORE (without DI) – Tight coupling, hard to test
public class FundTransferService {
    private AccountRepository accountRepository = new AccountRepository(); // manual creation

    public void transfer(Long fromId, Long toId, BigDecimal amt) {
        // business logic
    }
}

// AFTER (with DI) – Loosely coupled, easy to test
@Service
public class FundTransferService {
    private final AccountRepository accountRepository;

    // Spring injects the dependency through the constructor
    public FundTransferService(AccountRepository accountRepository) {
        this.accountRepository = accountRepository;
    }

    public void transfer(Long fromId, Long toId, BigDecimal amt) {
        // business logic
    }
}
```

Common Annotations

- **@Component** — marks a class as a Spring Bean.
- **@Service** — specialization of @Component for the service layer.
- **@Repository** — specialization of @Component for the DAO/persistence layer.
- **@Autowired** — not needed with constructor injection, but used for field/setter injection.
- **@Primary** — used to break ties when multiple candidate beans exist.

Common Mistakes

- Using **new** instead of letting Spring inject the dependency.
- Relying on field injection, which is hard to unit test.
- Forgetting @Service/@Component so the class is never picked up by component scanning.
- Having multiple candidate beans without @Primary or @Qualifier to disambiguate.



Fig 8. Dependency Injection explained with the restaurant analogy and a banking code example.

9. Observability in Spring Boot & Spring AI

Definition

Observability is the ability to understand what is happening **inside** a running system just from its external outputs — metrics, logs, and traces. Modern Spring Boot + Spring AI applications are no longer simple request/response systems; they call external APIs, databases, LLMs, and multi-step AI agent workflows, so answering questions like "why is this slow?" requires purpose-built instrumentation.

Questions Observability Must Answer

- Why is an API slow?
- Which service increased latency?
- How long did an AI model call take?
- Which agent step failed?
- How are tokens and external calls impacting performance?

Six Ways to Integrate Observability

- **Application-Level Instrumentation** — Spring Actuator, Micrometer, OpenTelemetry SDK, or vendor libraries (e.g. Datadog) added directly into the app for custom business metrics and LLM latency/token tracking.
- **Micrometer + OpenTelemetry Bridge** — Spring Boot's native metrics abstraction (Micrometer) bridges into OpenTelemetry, which then exports to Datadog / Prometheus / Grafana.
- **OpenTelemetry SDK** — a vendor-neutral way to collect metrics, logs, and distributed traces; instrument once, send anywhere.
- **Java Agent Approach** — attach a Datadog or OpenTelemetry Java agent for automatic instrumentation (HTTP calls, DB operations, JVM metrics, distributed traces) with minimal code changes.
- **Sidecar Pattern** — run an OpenTelemetry Collector sidecar beside the Spring Boot service for centralized configuration and independent scaling.
- **Kubernetes DaemonSet Pattern** — run OTel Collectors as a DaemonSet across every node for cluster-wide infrastructure visibility.

Hands-On Example — Custom Metrics with Micrometer

```
@Service
public class LlmAgentService {

    private final MeterRegistry registry;

    public LlmAgentService(MeterRegistry registry) {
        this.registry = registry;
    }

    public String callModel(String prompt) {
        Timer.Sample sample = Timer.start(registry);
        try {
            String response = llmClient.complete(prompt);
            registry.counter("llm.calls.success").increment();
            return response;
        } catch (Exception e) {
            registry.counter("llm.calls.failure").increment();
            throw e;
```

```
        } finally {
            sample.stop(registry.timer("llm.call.duration"));
        }
    }
}

# application.yml
management:
  endpoints:
    web:
      exposure:
        include: health,metrics,prometheus
  metrics:
    tags:
      application: spring-ai-service
```

Why it matters: Tracing every step (User Request → Spring Boot API → AI Agent Workflow → LLM Call → External Tools → Response) with latency, errors, tokens, cost, and dependencies turns an opaque AI pipeline into an explainable system.

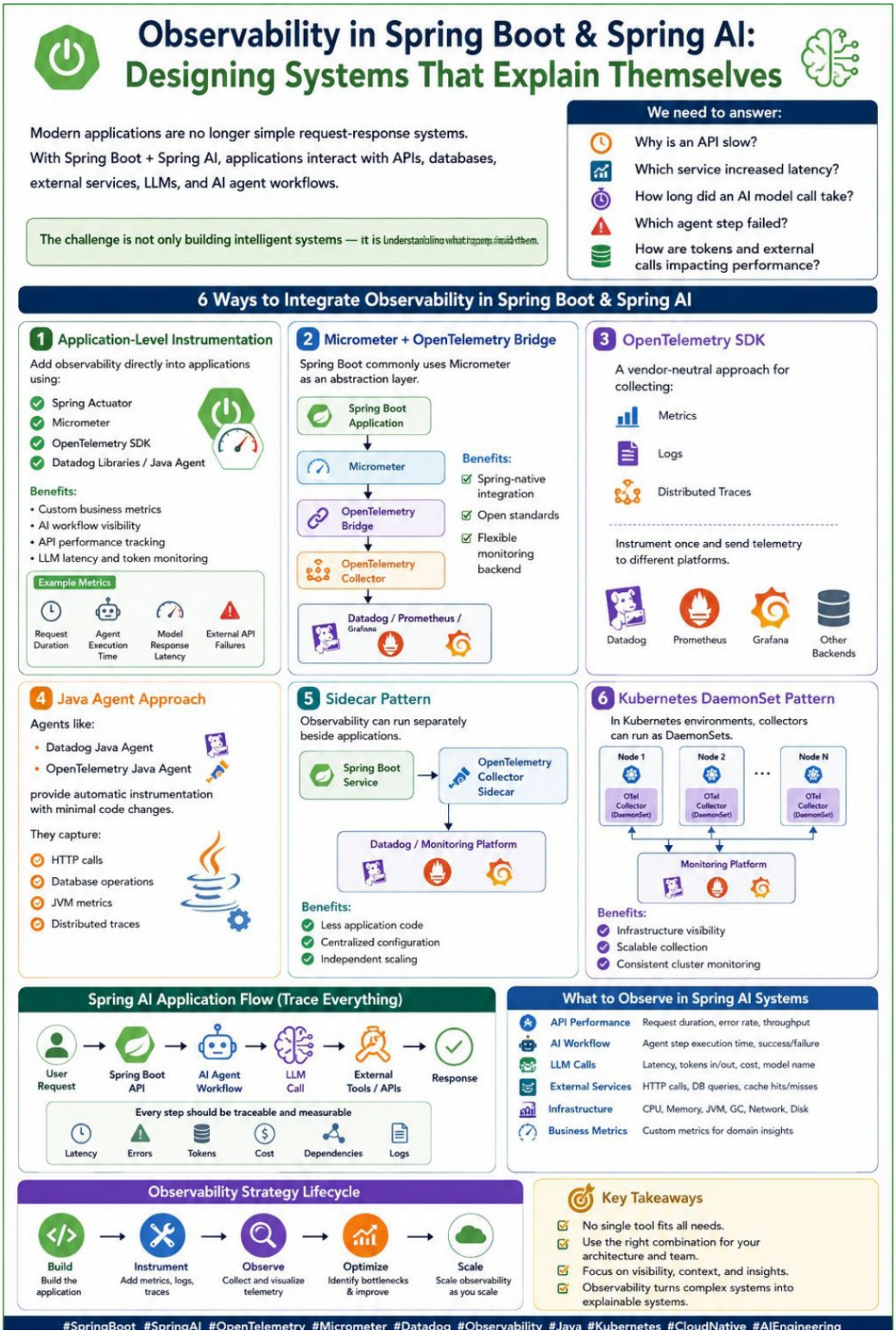


Fig 9. Six approaches to observability in Spring Boot and Spring AI systems.

10. CAP Theorem

Definition

In any distributed system you can only guarantee **two** of the following three properties at the same time, and only during a network partition:

- **Consistency (C)** — every read receives the latest write, or an error.
- **Availability (A)** — every request receives a (non-error) response.
- **Partition Tolerance (P)** — the system keeps working even if nodes can't talk to each other.

The Key Idea

Under normal conditions (no partition), a system can actually provide all three. It is only **during a network partition** that you are forced to sacrifice one of Consistency or Availability — because Partition Tolerance is not optional in a real distributed system.

The Three Trade-off Combinations

- **CA (Consistency + Availability)** — stays consistent and available but cannot tolerate partitions; the system may fail entirely during one. Example: traditional single-node RDBMS (MySQL, PostgreSQL).
- **CP (Consistency + Partition Tolerance)** — remains consistent and keeps working during a partition, but some requests may fail/timeout. Example: HBase, MongoDB (strict mode), ZooKeeper, etcd.
- **AP (Availability + Partition Tolerance)** — stays available and works during partitions, but may return stale or inconsistent data. Example: Cassandra, DynamoDB, CouchDB, Riak.

Real-World Design Decisions

- **Banking systems** need consistency more than availability → choose CP.
- **Chat / social feeds** need high availability and can tolerate some stale data → choose AP.
- **E-commerce order systems** usually need strong consistency for stock/payment correctness → choose CP.

Hands-On: Choosing a Data Store

```
// CP choice: bank account balance — never show stale/incorrect balance
@Transactional(isolation = Isolation.SERIALIZABLE)
public void withdraw(Long accountId, BigDecimal amount) {
    Account acc = accountRepository.findByIdForUpdate(accountId); // pessimistic lock
    if (acc.getBalance().compareTo(amount) < 0)
        throw new InsufficientFundsException();
    acc.setBalance(acc.getBalance().subtract(amount));
    accountRepository.save(acc);
}

// AP choice: social feed like counter — eventual consistency is fine
public void incrementLikeCounter(String postId) {
    cassandraTemplate.execute(
        "UPDATE post_likes SET count = count + 1 WHERE post_id = ?", postId);
    // Reads across replicas may briefly show different counts — acceptable trade-off
}
```

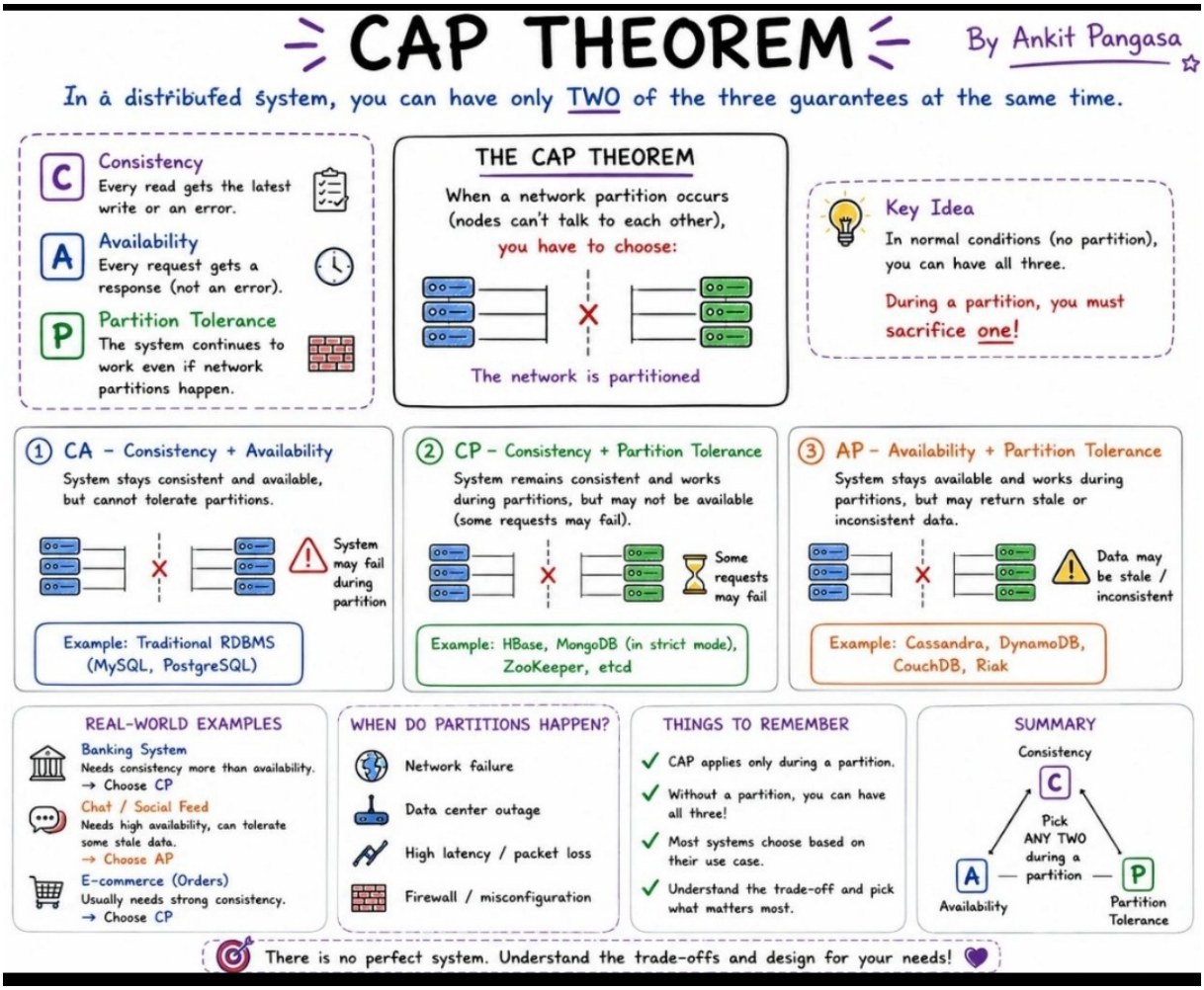


Fig 10. CAP Theorem — pick any two of Consistency, Availability, Partition Tolerance during a partition.

11. Spring Security — Internal Filter Chain

Definition

Spring Security intercepts every HTTP request through a chain of **Servlet Filters** before it reaches your controller. Each filter has one responsibility, and order matters.

The Filter Chain, Step by Step

- **SecurityContextPersistenceFilter** — loads the SecurityContext from the repository (if any) or creates an empty one for the request.
- **UsernamePasswordAuthenticationFilter** — handles login requests (e.g. /login); if this isn't a login request, it passes control to the next filter.
- **JWT / Custom AuthenticationFilter** — extracts the JWT from the Authorization header, validates the token, extracts the username, and sets Authentication in the SecurityContext if valid.
- **AuthenticationManager** — authenticates the user by delegating to one or more AuthenticationProvider(s), which typically use a UserDetailsService to load and validate credentials.
- **AuthorizationFilter (FilterSecurityInterceptor)** — checks whether the authenticated user is authorized to access the requested resource, based on URL patterns, HTTP methods, and roles/authorities.
- **Controller / Resource** — if authorized, the request finally reaches your controller and a response is returned.

What Happens Inside Each Stage

- **SecurityContext** — holds the Authentication for the current request; stored in a ThreadLocal.
- **Authentication** — a UsernamePasswordAuthenticationToken with principal (UserDetails), credentials, and authorities.
- **AuthenticationProvider** — uses UserDetailsService to load and validate the user.
- **Decision based on access rules** — URL patterns, HTTP methods, roles/authorities, and custom rules.
- **After request completion** — SecurityContextPersistenceFilter clears the SecurityContext so it doesn't leak between requests/threads.

Hands-On Example — JWT Filter

```
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
                                    FilterChain chain) throws IOException, ServletException {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            if (jwtService.isValid(token)) {
                String username = jwtService.extractUsername(token);
                UserDetails user = userDetailsService.loadUserByUsername(username);
                UsernamePasswordAuthenticationToken authToken =
                    new UsernamePasswordAuthenticationToken(user, null, user.getAuthorities());
                SecurityContextHolder.getContext().setAuthentication(authToken);
            }
        }
        chain.doFilter(req, res); // continue to AuthorizationFilter
    }
```

```
}

// SecurityConfig
http.authorizeHttpRequests(auth -> auth
        .requestMatchers("/api/**").authenticated()
        .anyRequest().permitAll())
    .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);
```



Fig 11. Spring Security filter chain — from HTTP request to controller.

12. Hibernate Entity Lifecycle

Definition

Hibernate manages the lifecycle of every entity object from creation to deletion, tracking its state inside the **Persistence Context** (the first-level cache) and synchronizing changes with the database.

The Four Entity States

- **Transient** — the entity is new and not associated with any Persistence Context; Hibernate does not track it. Example: *new Employee()*.
- **Persistent (Managed)** — the entity is associated with a Persistence Context; Hibernate tracks all changes and automatically synchronizes them with the DB.
- **Detached** — the entity was persistent but is no longer associated with a Persistence Context (after detach/clear/close); changes are no longer tracked.
- **Removed** — the entity is marked for deletion and will be deleted from the DB on flush/commit.

Methods to Change State

- **persist(entity)** — makes a transient entity persistent (generates INSERT); throws an error if the entity already exists.
- **merge(entity)** — copies the state of a detached entity into a managed instance and returns that managed instance (generates INSERT or UPDATE).
- **update(entity)** — reattaches a detached entity if not already in the context; throws if the entity already exists in the Persistence Context (Hibernate Session API specific).
- **remove(entity)** — marks the entity for deletion.
- **detach(entity)** — detaches the entity from the context.
- **refresh(entity)** — reloads the entity's state from the database, discarding in-memory changes.

Dirty Checking & flush() vs commit()

- **Dirty Checking** — Hibernate automatically detects changes in managed entities by comparing current state against the snapshot taken when the entity was loaded.
- **flush()** — synchronizes changes from the Persistence Context to the DB by executing SQL (INSERT/UPDATE/DELETE); it does NOT commit the transaction.
- **commit()** — triggers a flush if needed, then commits the transaction, making changes permanent and ending the transaction.

Hands-On Example

```
@Transactional
public void demoLifecycle() {
    // TRANSIENT
    Employee emp = new Employee("Asha", "Engineering");

    // PERSISTENT (managed) - INSERT generated on flush/commit
    entityManager.persist(emp);

    emp.setDepartment("Platform");   // dirty checking detects this change automatically
    // no explicit save() call needed - Hibernate tracks and syncs it

    entityManager.flush();           // SQL UPDATE executed now, transaction still open
```

```
        entityManager.detach(emp);       // DETACHED - changes no longer tracked
        emp.setDepartment("Ignored");    // this change will NOT be persisted

        Employee managed = entityManager.merge(emp); // re-attach as managed, may generate UPDATE

        entityManager.remove(managed);   // REMOVED - marked for deletion
        // DELETE executed on commit
    }
```

Why it matters: Understanding these states explains subtle bugs — e.g. modifying a detached entity silently does nothing until it's merged back into a Persistence Context.

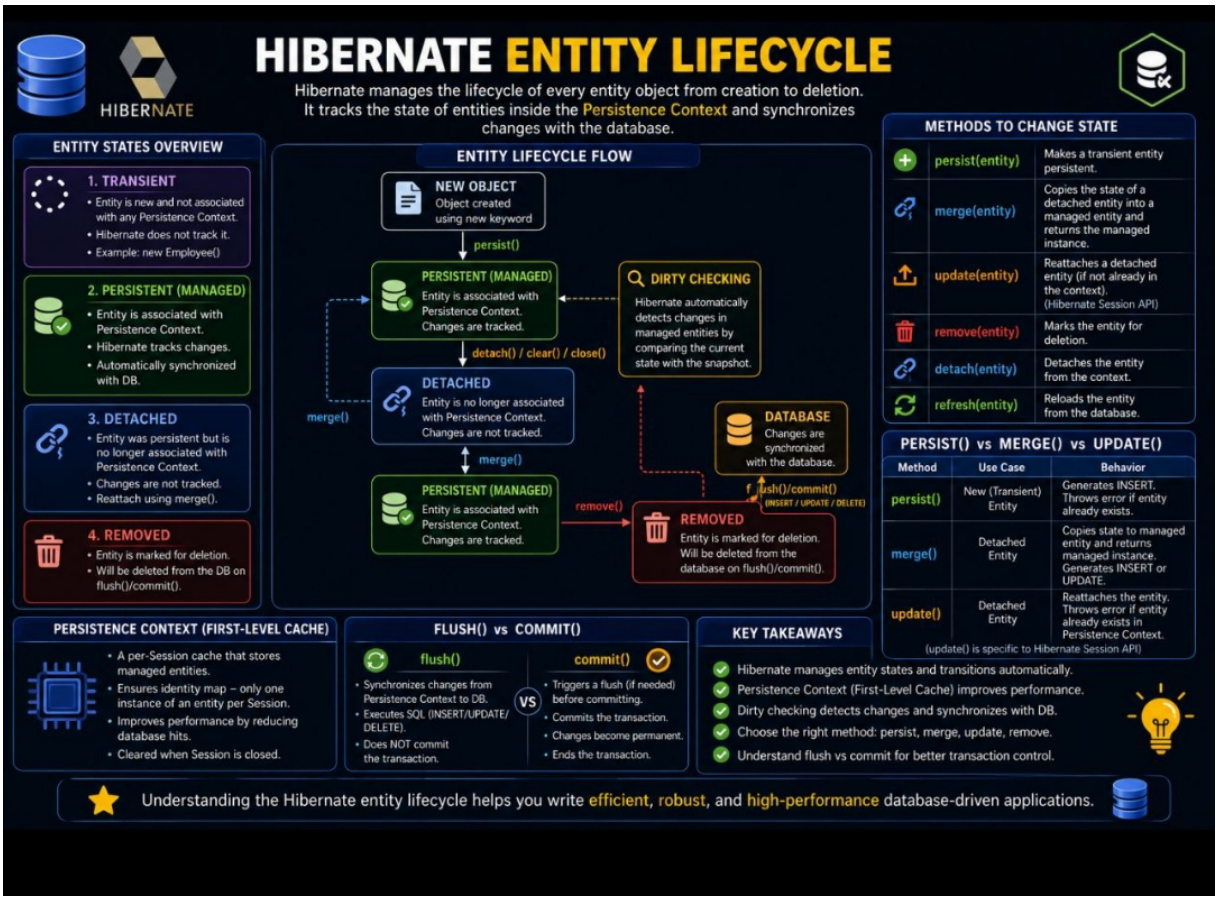


Fig 12. Hibernate Entity Lifecycle — Transient, Persistent, Detached, and Removed states.

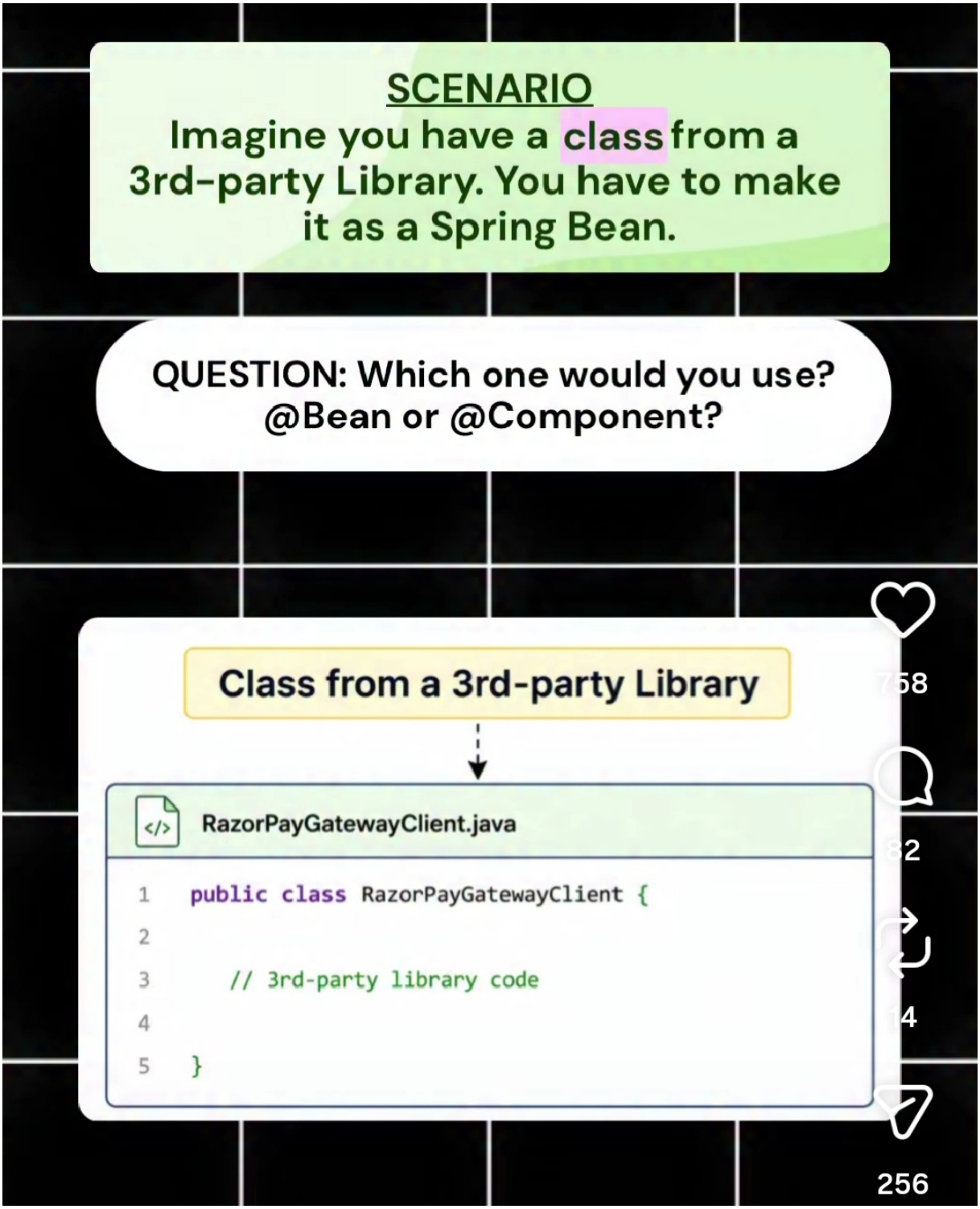
Part 2
Hands-On Microservice Solutions

Complete, runnable Spring Boot solutions to the 3 scenario-based questions shown in the source screenshots — entities, config, service layer, controllers, and test commands.

13. Registering a Third-Party Class as a Spring Bean

Scenario

You have a class from a third-party library — **RazorPayGatewayClient.java** — and you need to make it a Spring Bean. **Question: Which one would you use — @Bean or @Component?**



Original scenario screenshot.

Answer

Use **@Bean** inside a **@Configuration** class. @Component only works when you can add the annotation to the class itself, which requires owning the source code. Since RazorPayGatewayClient comes from a third-party JAR, you cannot edit or annotate it — so you manually construct and register it as a bean using a @Bean-annotated factory method instead.

Hands-On Microservice Solution

Project layout (payment-service, a Spring Boot microservice):

```
payment-service/
 ■■■ src/main/java/com/bank/payment/
      ■■■ PaymentServiceApplication.java
      ■■■ config/PaymentGatewayConfig.java
      ■■■ client/RazorPayGatewayClient.java   (3rd-party, cannot be modified)
      ■■■ service/PaymentService.java
      ■■■ controller/PaymentController.java
 ■■■ src/main/resources/application.yml
```

Step 1 — The third-party class (as-is, cannot be annotated):

```
package com.thirdparty.razorpay;

public class RazorPayGatewayClient {

    private final String apiKey;
    private final String apiSecret;

    public RazorPayGatewayClient(String apiKey, String apiSecret) {
        this.apiKey = apiKey;
        this.apiSecret = apiSecret;
    }

    public String charge(String orderId, long amountPaise) {
        // 3rd-party library network call (simulated)
        return "txn_" + orderId + "_" + System.currentTimeMillis();
    }
}
```

Step 2 — Register it as a Bean via @Configuration + @Bean:

```
package com.bank.payment.config;

import com.thirdparty.razorpay.RazorPayGatewayClient;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class PaymentGatewayConfig {

    @Bean
    public RazorPayGatewayClient razorPayGatewayClient(
            @Value("${razorpay.api.key}") String apiKey,
            @Value("${razorpay.api.secret}") String apiSecret) {
        return new RazorPayGatewayClient(apiKey, apiSecret);
    }
}
```

Step 3 — Inject and use it like any other Spring bean:

```
package com.bank.payment.service;

import com.thirdparty.razorpay.RazorPayGatewayClient;
import org.springframework.stereotype.Service;

@Service
public class PaymentService {
```

```
    private final RazorPayGatewayClient gatewayClient;

    // Constructor injection - Spring supplies the bean created in PaymentGatewayConfig
    public PaymentService(RazorPayGatewayClient gatewayClient) {
        this.gatewayClient = gatewayClient;
    }

    public String pay(String orderId, long amountPaise) {
        return gatewayClient.charge(orderId, amountPaise);
    }
}

// application.yml
razorpay:
  api:
    key: rzp_test_XXXXXXXX
    secret: ${RAZORPAY_SECRET}   // pulled from environment variable
```

Step 4 — Expose it through a REST controller:

```
@RestController
@RequestMapping("/api/payments")
public class PaymentController {

    private final PaymentService paymentService;

    public PaymentController(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    @PostMapping("/{orderId}/charge")
    public ResponseEntity<String> charge(@PathVariable String orderId,
                                         @RequestParam long amountPaise) {
        return ResponseEntity.ok(paymentService.pay(orderId, amountPaise));
    }
}

// Test with curl
// curl -X POST "http://localhost:8080/api/payments/ORD123/charge?amountPaise=50000"
```

Result: RazorPayGatewayClient is now fully managed by Spring — it can be injected anywhere, swapped out with a mock in unit tests via a test @Configuration, and its lifecycle (creation/destruction) is handled by the IoC container, even though its source code is untouched.

14. Role-Based Access Control for a Banking Microservice

Scenario

Consider a Banking Application where: **Consumers** can VIEW loan applications; **Employees** can CREATE & VIEW; **Managers** can CREATE, VIEW & APPROVE. **Question: How would you implement this in Spring Boot?**



Original scenario screenshot.

Answer

Model each permission tier as a **role** (ROLE_CONSUMER, ROLE_EMPLOYEE, ROLE_MANAGER), issue the role(s) inside a JWT at login, then enforce access with Spring Security's **method-level security** (@PreAuthorize) on the service/controller layer, backed by a URL-level filter chain as a second line of defense.

Hands-On Microservice Solution

Project layout (loan-service):

```
loan-service/
 ■■■ src/main/java/com/bank/loan/
      ■■■ LoanServiceApplication.java
      ■■■ config/SecurityConfig.java
      ■■■ security/JwtAuthenticationFilter.java
      ■■■ model/Loan.java
      ■■■ repository/LoanRepository.java
      ■■■ service/LoanService.java
      ■■■ controller/LoanController.java
```

Step 1 — Enable method security and define the filter chain:

```
@Configuration
@EnableMethodSecurity(prePostEnabled = true)   // turns on @PreAuthorize
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtFilter;

    public SecurityConfig(JwtAuthenticationFilter jwtFilter) {
        this.jwtFilter = jwtFilter;
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(HttpMethod.GET,  "/api/loans/**").hasAnyRole("CONSUMER","EMPLOYEE","M
                .requestMatchers(HttpMethod.POST, "/api/loans").hasAnyRole("EMPLOYEE","MANAGER")
                .requestMatchers(HttpMethod.POST, "/api/loans/*/approve").hasRole("MANAGER")
                .anyRequest().authenticated())
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}
```

Step 2 — JWT filter extracts the role(s) and populates the SecurityContext:

```
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain
            throws ServletException, IOException {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            if (jwtService.isValid(token)) {
                String username = jwtService.extractUsername(token);
                List<GrantedAuthority> authorities = jwtService.extractRoles(token).stream()
                        .map(r -> new SimpleGrantedAuthority("ROLE_" + r))
                        .collect(Collectors.toList());
                var authToken = new UsernamePasswordAuthenticationToken(username, null, authorities);
                SecurityContextHolder.getContext().setAuthentication(authToken);
```

```
            }
        }
        chain.doFilter(req, res);
    }
}
```

Step 3 — Entity + repository:

```
@Entity
public class Loan {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String applicantUsername;
    private BigDecimal amount;
    @Enumerated(EnumType.STRING)
    private LoanStatus status;   // PENDING, APPROVED, REJECTED
    // getters/setters
}

public interface LoanRepository extends JpaRepository<Loan, Long> {
    List<Loan> findByApplicantUsername(String username);
}
```

Step 4 — Service layer with method-level enforcement as defense-in-depth:

```
@Service
public class LoanService {
    private final LoanRepository loanRepository;

    public LoanService(LoanRepository loanRepository) { this.loanRepository = loanRepository; }

    @PreAuthorize("hasAnyRole('CONSUMER','EMPLOYEE','MANAGER')")
    public List<Loan> viewLoans(String username, boolean isPrivileged) {
        return isPrivileged ? loanRepository.findAll()
                            : loanRepository.findByApplicantUsername(username);
    }

    @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
    public Loan createLoan(Loan loan) {
        loan.setStatus(LoanStatus.PENDING);
        return loanRepository.save(loan);
    }

    @PreAuthorize("hasRole('MANAGER')")
    public Loan approveLoan(Long id) {
        Loan loan = loanRepository.findById(id).orElseThrow();
        loan.setStatus(LoanStatus.APPROVED);
        return loanRepository.save(loan);
    }
}
```

Step 5 — Controller + test commands:

```
@RestController
@RequestMapping("/api/loans")
public class LoanController {
    private final LoanService loanService;
    public LoanController(LoanService loanService) { this.loanService = loanService; }

    @GetMapping
    public List<Loan> viewLoans(Authentication auth) {
        boolean privileged = auth.getAuthorities().stream()
            .anyMatch(a -> a.getAuthority().equals("ROLE_EMPLOYEE") || a.getAuthority().equals("ROLE_M
        return loanService.viewLoans(auth.getName(), privileged);
    }

    @PostMapping
    public Loan create(@RequestBody Loan loan) { return loanService.createLoan(loan); }

    @PostMapping("/{id}/approve")
```

```
    public Loan approve(@PathVariable Long id) { return loanService.approveLoan(id); }
}

// Test as Consumer (view only)
// curl -H "Authorization: Bearer <consumer-jwt>" http://localhost:8081/api/loans

// Test as Employee (create)
// curl -X POST -H "Authorization: Bearer <employee-jwt>" -H "Content-Type: application/json" \
//      -d '{"applicantUsername":"asha","amount":50000}' http://localhost:8081/api/loans

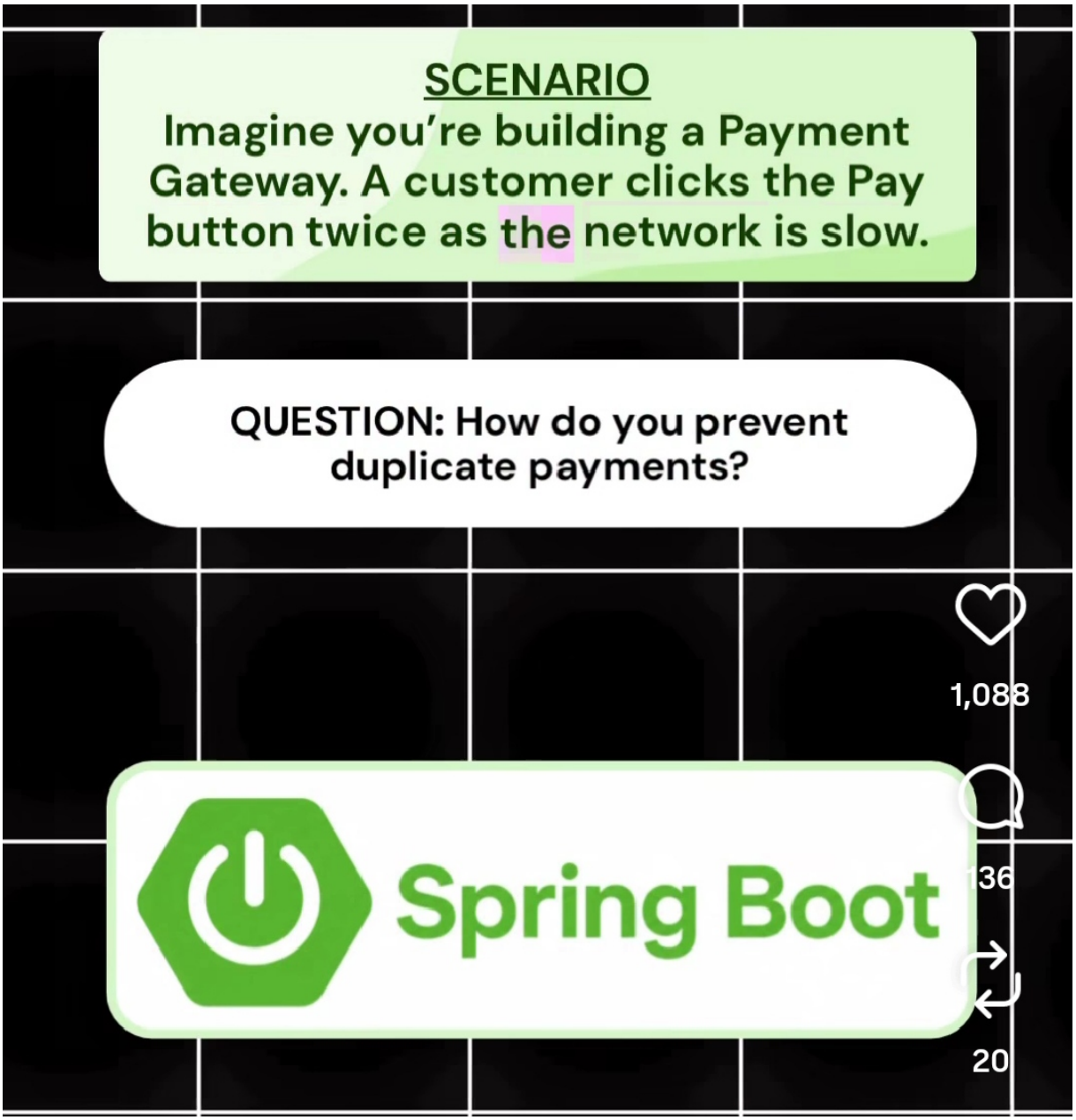
// Test as Manager (approve) -> 403 Forbidden if attempted by Employee/Consumer
// curl -X POST -H "Authorization: Bearer <manager-jwt>" http://localhost:8081/api/loans/1/approve
```

Result: Access rules are enforced at two layers — the HTTP filter chain (coarse-grained URL rules) and @PreAuthorize on the service (fine-grained, testable, and safe even if a new controller bypasses the URL rule).

15. Preventing Duplicate Payments in a Payment Gateway Microservice

Scenario

You're building a Payment Gateway. A customer clicks the Pay button twice because the network is slow.
Question: How do you prevent duplicate payments?



Original scenario screenshot.

Answer

Require the client to send a unique **Idempotency-Key** header with every payment attempt (generated once, reused on retries). The server persists the key with the transaction result the first time, backed by a **unique database constraint**, and simply returns the stored result on any repeat request with the same key — instead of charging the customer again.

Hands-On Microservice Solution

Project layout (payment-gateway-service):

```
payment-gateway-service/
 ■■■ src/main/java/com/bank/gateway/
      ■■■ PaymentGatewayApplication.java
      ■■■ model/PaymentRecord.java
      ■■■ repository/PaymentRecordRepository.java
      ■■■ service/PaymentGatewayService.java
      ■■■ controller/PaymentGatewayController.java
 ■■■ src/main/resources/db/migration/V1__create_payments.sql
```

Step 1 — Database table with a unique constraint (the real safety net):

```
-- V1__create_payments.sql (Flyway migration)
CREATE TABLE payments (
    id BIGSERIAL PRIMARY KEY,
    idempotency_key VARCHAR(64) NOT NULL UNIQUE,
    order_id VARCHAR(64) NOT NULL,
    amount_paise BIGINT NOT NULL,
    status VARCHAR(20) NOT NULL,
    transaction_ref VARCHAR(64),
    created_at TIMESTAMP DEFAULT now()
);
```

Step 2 — Entity + repository:

```
@Entity
@Table(name = "payments")
public class PaymentRecord {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "idempotency_key", unique = true, nullable = false)
    private String idempotencyKey;

    private String orderId;
    private long amountPaise;

    @Enumerated(EnumType.STRING)
    private PaymentStatus status;   // SUCCESS, FAILED

    private String transactionRef;
    // getters/setters
}

public interface PaymentRecordRepository extends JpaRepository<PaymentRecord, Long> {
    Optional<PaymentRecord> findByIdempotencyKey(String idempotencyKey);
}
```

Step 3 — Service layer: check first, catch race conditions second:

```
@Service
public class PaymentGatewayService {

    private final PaymentRecordRepository repository;
    private final RazorPayGatewayClient gatewayClient;

    public PaymentGatewayService(PaymentRecordRepository repository, RazorPayGatewayClient gatewayClie
        this.repository = repository;
        this.gatewayClient = gatewayClient;
    }

    @Transactional
    public PaymentRecord processPayment(String idempotencyKey, String orderId, long amountPaise) {

        // 1) Fast path: already processed -> return the SAME stored result
        Optional<PaymentRecord> existing = repository.findByIdempotencyKey(idempotencyKey);
        if (existing.isPresent()) {
            return existing.get();
        }
```

```
        try {
            // 2) Not seen before -> actually charge, then persist atomically
            String txnRef = gatewayClient.charge(orderId, amountPaise);

            PaymentRecord record = new PaymentRecord();
            record.setIdempotencyKey(idempotencyKey);
            record.setOrderId(orderId);
            record.setAmountPaise(amountPaise);
            record.setStatus(PaymentStatus.SUCCESS);
            record.setTransactionRef(txnRef);
            return repository.save(record);   // unique constraint protects against races

        } catch (DataIntegrityViolationException raceCondition) {
            // 3) Two near-simultaneous requests with the same key both passed step 1 ->
            //    the DB unique constraint rejected the second INSERT. Return the winner's result.
            return repository.findByIdempotencyKey(idempotencyKey).orElseThrow();
        }
    }
}
```

Step 4 — Controller requiring the Idempotency-Key header:

```
@RestController
@RequestMapping("/api/gateway/payments")
public class PaymentGatewayController {

    private final PaymentGatewayService paymentGatewayService;

    public PaymentGatewayController(PaymentGatewayService paymentGatewayService) {
        this.paymentGatewayService = paymentGatewayService;
    }

    @PostMapping
    public ResponseEntity<PaymentRecord> pay(
            @RequestHeader("Idempotency-Key") String idempotencyKey,
            @RequestParam String orderId,
            @RequestParam long amountPaise) {

        PaymentRecord result = paymentGatewayService.processPayment(idempotencyKey, orderId, amountPai
        return ResponseEntity.ok(result);
    }
}
```

Step 5 — Client-side: generate the key once, reuse on retry:

```
// Frontend / calling client (pseudo-JS)
const idempotencyKey = crypto.randomUUID();   // generated ONCE when the user clicks Pay

async function pay(orderId, amountPaise, retries = 3) {
    for (let attempt = 0; attempt < retries; attempt++) {
        try {
            return await fetch(`/api/gateway/payments?orderId=${orderId}&amountPaise=${amountPaise}`,
                method: "POST",
                headers: { "Idempotency-Key": idempotencyKey }   // SAME key every retry
            });
        } catch (networkError) {
            await sleep(500 * (attempt + 1));   // retry on slow/failed network
        }
    }
}

// Test with curl - run this twice with the SAME key; only ONE charge happens
// curl -X POST -H "Idempotency-Key: 7c9e6679-7425-40de-944b-e07fc1f90ae7" \
//      "http://localhost:8082/api/gateway/payments?orderId=ORD123&amountPaise=50000"
// curl -X POST -H "Idempotency-Key: 7c9e6679-7425-40de-944b-e07fc1f90ae7" \
//      "http://localhost:8082/api/gateway/payments?orderId=ORD123&amountPaise=50000"
// -> Both calls return the SAME transaction_ref; the customer is charged only once.
```

Result: The double-click / slow-network scenario is neutralized: the first request performs the charge, the second (duplicate) request is short-circuited and returns the identical stored result — no second charge, no race condition, and the client can safely retry on any network failure.

16. System Architecture Diagrams (as seen in Image 9)

About This Image

This screenshot is different from the others in your set — it's a paused frame of a career-story reel ("I was laid off by Atlassian") rather than a clean, purpose-made infographic. In the top-right corner of the paused video there is a small **system/cloud architecture diagram** — boxes connected by arrows in a node-and-flow layout, typical of an infrastructure or service-architecture diagram. **Honesty note:** even after cropping and enlarging it, the text labels inside the boxes are too low-resolution to read reliably, so I won't guess at the specific service names. What follows instead is a faithful, hands-on explanation of *what this class of diagram is and represents*, plus a fully worked Spring Boot microservices example of the same kind of diagram, which you can compare against the original.

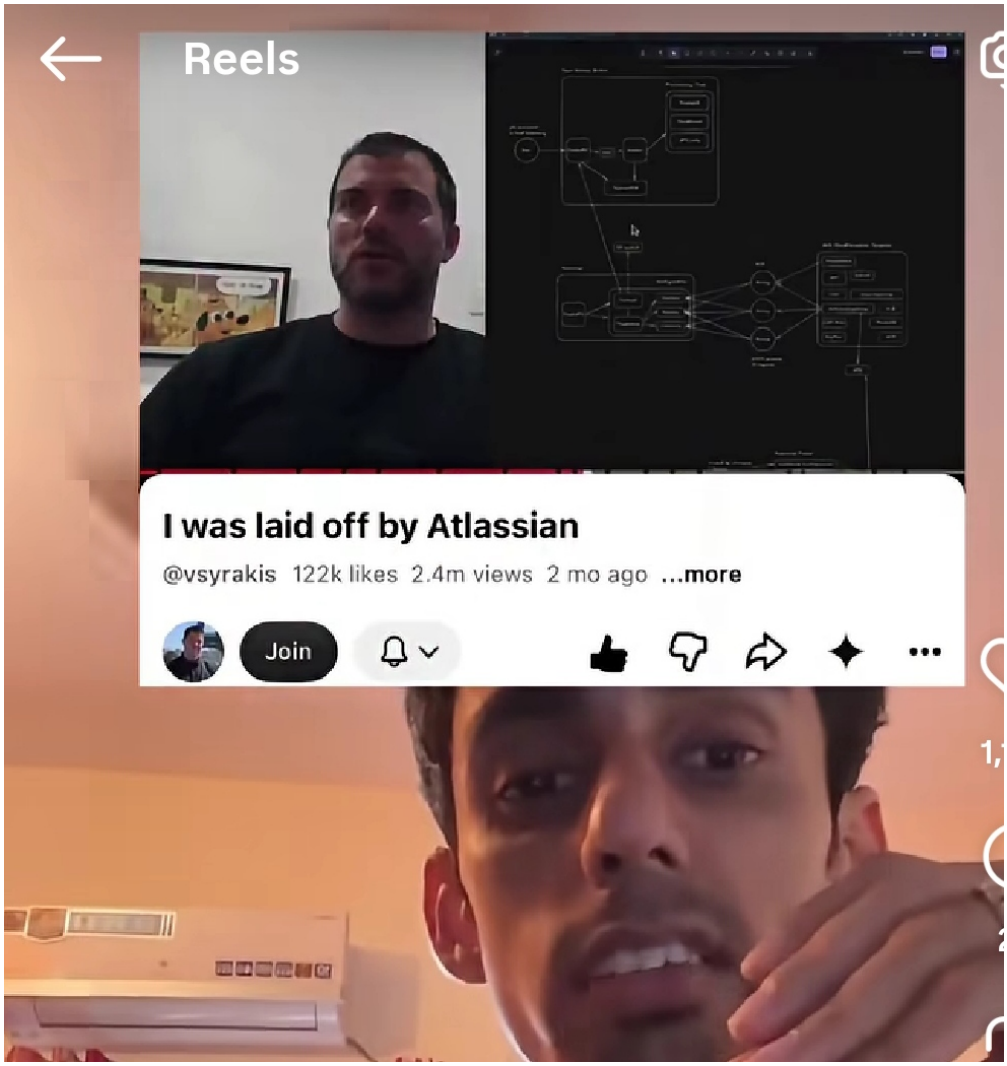


Fig 16. Original screenshot — architecture diagram visible in the paused video (top-right).

Definition

A **system (or cloud) architecture diagram** is a visual representation of a software system's components — services, databases, queues, gateways, external APIs — and the connections between them (data flow, dependencies, and calls). It typically follows a **node-and-edge** layout: boxes represent components; arrows represent requests, events, or data moving between them. These diagrams are used to communicate design decisions, onboard new engineers, and plan infrastructure-as-code deployments (e.g. AWS CloudFormation / Terraform).

What These Diagrams Usually Contain

- **Entry points** — an API Gateway or Load Balancer receiving client requests.
- **Services/nodes** — individual microservices, each with a single responsibility.
- **Data stores** — databases, caches, or object storage attached to specific services.
- **Messaging** — queues or event buses (Kafka, SQS, RabbitMQ) connecting services asynchronously.
- **External dependencies** — third-party APIs, payment gateways, identity providers.
- **Direction of arrows** — showing request/response flow or event publication vs. consumption.

Hands-On: Building the Same Kind of Diagram for a Spring Boot Microservices System

Below is a worked example of exactly this style of diagram — drawn as code using Mermaid — for a typical order-processing microservices system, followed by the Spring Boot / Spring Cloud building blocks that implement each box.

````
```mermaid
flowchart LR
    Client -->|HTTPS| Gateway[API Gateway]
    Gateway --> OrderSvc[Order Service]
    Gateway --> UserSvc[User Service]
    OrderSvc -->|publish OrderCreated| Broker[(Kafka)]
    Broker --> PaymentSvc[Payment Service]
    Broker --> NotifySvc[Notification Service]
    OrderSvc --> OrderDB[(Orders DB)]
    PaymentSvc --> PaymentDB[(Payments DB)]
    PaymentSvc --> Gateway3rdParty[3rd-Party Payment API]
```
````

Mapping each box to real Spring Boot components:

- **API Gateway** → Spring Cloud Gateway, routing requests to the correct downstream service.
- **Order Service / User Service** → independent Spring Boot applications, each with its own database (database-per-service pattern).
- **Kafka broker** → Spring Kafka producer/consumer for asynchronous, decoupled communication.
- **Orders DB / Payments DB** → Spring Data JPA repositories, one schema per service.
- **3rd-Party Payment API** → a REST client (e.g. RestClient/WebClient) calling out, wrapped as a Spring Bean (see Section 2 / 13 of this document).

```
// Order Service publishes an event instead of calling Payment Service directly
@Service
public class OrderService {
    private final KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate;
    private final OrderRepository orderRepository;

    public OrderService(KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate,
                        OrderRepository orderRepository) {
        this.kafkaTemplate = kafkaTemplate;
        this.orderRepository = orderRepository;
    }

    @Transactional
    public Order placeOrder(OrderRequest request) {
        Order order = orderRepository.save(Order.from(request));
        kafkaTemplate.send("order-created-topic",
                new OrderCreatedEvent(order.getId(), order.getAmount(), order.getUserId()));
        return order;
    }
}
```

```
// Payment Service listens and reacts independently
@Service
public class PaymentEventListener {
    private final PaymentGatewayService paymentGatewayService;

    @KafkaListener(topics = "order-created-topic", groupId = "payment-service")
    public void onOrderCreated(OrderCreatedEvent event) {
        paymentGatewayService.processPayment(
                "order-" + event.orderId(),          // idempotency key derived from order id
                event.orderId().toString(),
                event.amountPaise());
    }
}

// application.yml (per service - API Gateway routing example)
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://ORDER-SERVICE
          predicates:
            - Path=/api/orders/**
        - id: user-service
          uri: lb://USER-SERVICE
          predicates:
            - Path=/api/users/**
```

Why it matters: Drawing the architecture first — even as a quick Mermaid/draw.io diagram — clarifies service boundaries and communication style (sync REST vs. async events) *before* writing code. It's also exactly the kind of diagram engineers keep on hand (as in Image 9) to explain a system they built, during interviews, design reviews, or career retrospectives.

If you'd like an exact match to the original diagram rather than this representative example, share a higher-resolution screenshot or the original video/source, and I'll redo this section against the actual labels.

17. Functional Interfaces — Extended Hands-On (Spring Boot Microservice)

Definition (Recap)

A **Functional Interface** is an interface with exactly one abstract method (SAM), enabling it to be implemented inline with a **lambda expression** instead of a full class. Java's **java.util.function** package ships ready-made ones (Predicate, Function, Consumer, Supplier, BiFunction, UnaryOperator) so you rarely need to declare your own.

Hands-On: A Real Transaction-Filtering Microservice

Below is a complete **transaction-report-service** that uses five different functional interfaces together to filter, transform, validate, and summarize banking transactions — the kind of logic that shows up constantly in real Spring Boot services.

```
// Domain model
public record Transaction(String id, String type, BigDecimal amount, String status) {}

@Service
public class TransactionReportService {

    private final TransactionRepository repository;

    public TransactionReportService(TransactionRepository repository) {
        this.repository = repository;
    }

    // Predicate<T> — represents a condition (returns boolean)
    private static final Predicate<Transaction> IS_SUCCESSFUL =
            t -> "SUCCESS".equals(t.status());

    private static final Predicate<Transaction> IS_HIGH_VALUE =
            t -> t.amount().compareTo(BigDecimal.valueOf(100000)) > 0;

    // Function<T,R> — transforms one type into another
    private static final Function<Transaction, TransactionSummary> TO_SUMMARY =
            t -> new TransactionSummary(t.id(), t.type(), t.amount());

    // Consumer<T> — performs an action, returns nothing
    private static final Consumer<Transaction> LOG_TRANSACTION =
            t -> log.info("Processed transaction {} of type {}", t.id(), t.type());

    // Supplier<T> — takes no input, returns a value
    private final Supplier<String> reportIdGenerator =
            () -> "RPT-" + UUID.randomUUID();

    // BiFunction<T,U,R> — takes two inputs, returns a result
    private static final BiFunction<BigDecimal, BigDecimal, BigDecimal> APPLY_FX_RATE =
            (amount, rate) -> amount.multiply(rate).setScale(2, RoundingMode.HALF_UP);

    public ReportResult generateHighValueReport() {
        String reportId = reportIdGenerator.get();

        List<TransactionSummary> summaries = repository.findAll().stream()
                .filter(IS_SUCCESSFUL.and(IS_HIGH_VALUE))   // combine predicates
                .peek(LOG_TRANSACTION)                       // side-effect via Consumer
                .map(TO_SUMMARY)                             // transform via Function
                .collect(Collectors.toList());

        BigDecimal totalInUsd = summaries.stream()
                .map(s -> APPLY_FX_RATE.apply(s.amount(), BigDecimal.valueOf(0.012)))
                .reduce(BigDecimal.ZERO, BigDecimal::add);
```

```
            return new ReportResult(reportId, summaries, totalInUsd);
        }
    }
```

Hands-On: A Custom Functional Interface for Business Rules

When the built-in interfaces don't fit, define your own — this is common for pluggable business rules, such as validating a loan application with different strategies per bank product.

```
    @FunctionalInterface
    public interface LoanValidationRule {
        ValidationResult validate(LoanApplication application);

        // default methods let you compose custom functional interfaces, just like Predicate.and()
        default LoanValidationRule and(LoanValidationRule other) {
            return app -> {
                ValidationResult result = this.validate(app);
                return result.isValid() ? other.validate(app) : result;
            };
        }
    }

    @Service
    public class LoanValidationService {

        private final LoanValidationRule minimumIncomeRule =
                app -> app.getMonthlyIncome().compareTo(BigDecimal.valueOf(30000)) >= 0
                        ? ValidationResult.ok()
                        : ValidationResult.fail("Income below minimum threshold");

        private final LoanValidationRule creditScoreRule =
                app -> app.getCreditScore() >= 650
                        ? ValidationResult.ok()
                        : ValidationResult.fail("Credit score too low");

        public ValidationResult validate(LoanApplication application) {
            LoanValidationRule combinedRule = minimumIncomeRule.and(creditScoreRule);
            return combinedRule.validate(application);   // lambdas composed and executed
        }
    }
```

Unit Test (Hands-On Proof It Works)

```
    @Test
    void highValueSuccessfulTransactionsAreIncluded() {
        Transaction t1 = new Transaction("t1", "TRANSFER", new BigDecimal("150000"), "SUCCESS");
        Transaction t2 = new Transaction("t2", "TRANSFER", new BigDecimal("500"),    "SUCCESS");
        when(repository.findAll()).thenReturn(List.of(t1, t2));

        ReportResult result = transactionReportService.generateHighValueReport();

        assertThat(result.summaries()).hasSize(1);
        assertThat(result.summaries().get(0).id()).isEqualTo("t1");
    }
```

Why it matters: Functional interfaces let you pass behavior (not just data) into methods — filters, validators, transformers — making Stream pipelines, validation chains, and callback-style APIs concise and composable instead of requiring a new class per behavior.

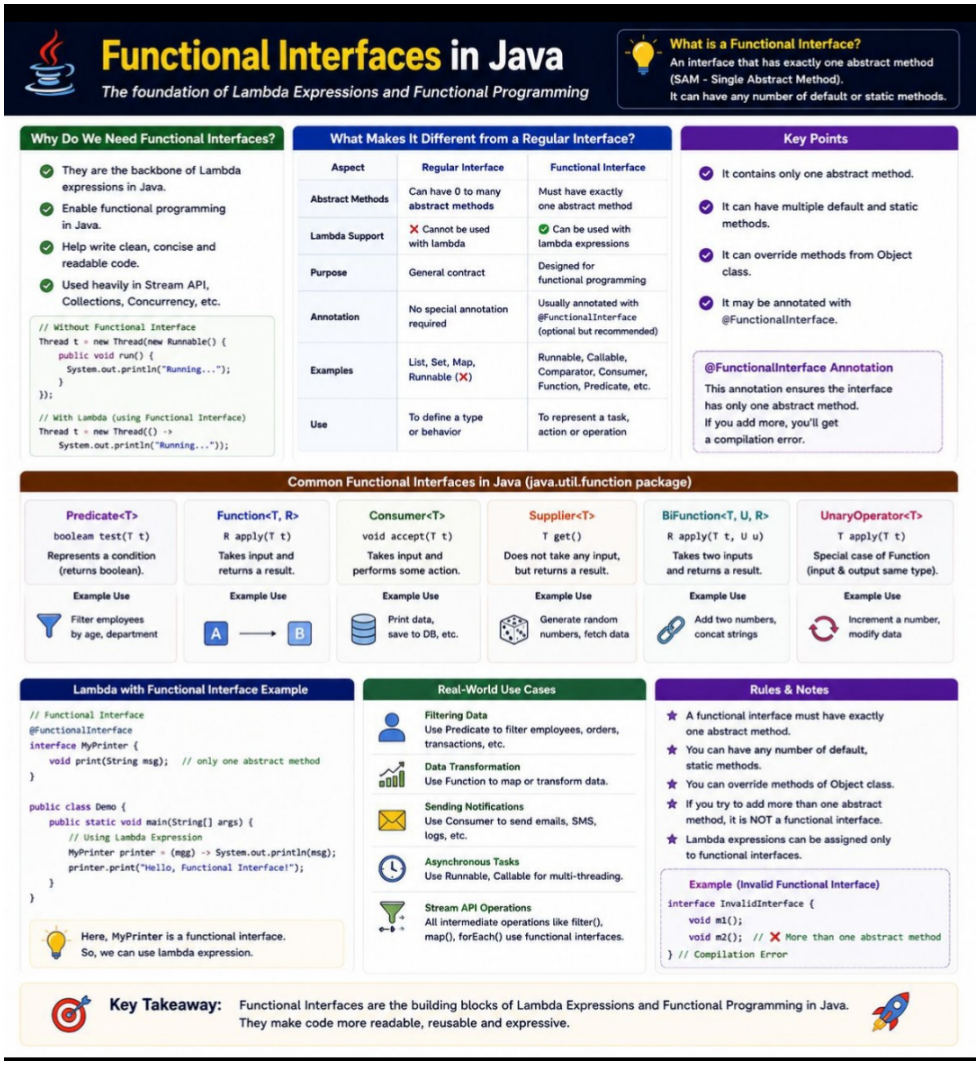


Fig 17. Functional Interfaces reference (original infographic).

18. Dependency Injection — Extended Hands-On (Spring Boot Microservice)

Definition (Recap)

Dependency Injection (DI) is the pattern where the Spring IoC Container creates an object's dependencies and supplies them to it, rather than the object constructing them itself. This inverts control of object creation from your code to the framework — hence Inversion of Control.

Hands-On: All Three Injection Styles, Compared

```
// 1) FIELD INJECTION - works, but hard to test and hides required dependencies
@Service
public class NotificationServiceFieldInjection {
    @Autowired
    private EmailClient emailClient;     // cannot be final; no compile-time guarantee it's set
}

// 2) SETTER INJECTION - optional dependencies, but mutable after construction
@Service
public class NotificationServiceSetterInjection {
    private SmsClient smsClient;

    @Autowired
    public void setSmsClient(SmsClient smsClient) {
        this.smsClient = smsClient;
    }
}

// 3) CONSTRUCTOR INJECTION - RECOMMENDED: immutable, testable, dependencies explicit
@Service
public class NotificationService {
    private final EmailClient emailClient;
    private final SmsClient smsClient;

    // @Autowired is optional here (Spring 4.3+) since there's only one constructor
    public NotificationService(EmailClient emailClient, SmsClient smsClient) {
        this.emailClient = emailClient;
        this.smsClient = smsClient;
    }

    public void notifyUser(String userId, String message) {
        emailClient.send(userId, message);
        smsClient.send(userId, message);
    }
}
```

Hands-On: Resolving Multiple Candidate Beans (@Qualifier / @Primary)

```
public interface PaymentGateway {
    String charge(String orderId, long amountPaise);
}

@Component("razorPayGateway")
public class RazorPayGateway implements PaymentGateway {
    public String charge(String orderId, long amountPaise) { return "razorpay-txn-" + orderId; }
}

@Component("stripeGateway")
@Primary                                    // used when no @Qualifier is specified
public class StripeGateway implements PaymentGateway {
    public String charge(String orderId, long amountPaise) { return "stripe-txn-" + orderId; }
}

@Service
```

```
public class CheckoutService {
    private final PaymentGateway paymentGateway;

    // Explicitly pick the non-default bean by qualifier
    public CheckoutService(@Qualifier("razorPayGateway") PaymentGateway paymentGateway) {
        this.paymentGateway = paymentGateway;
    }
}
```

Hands-On: Why Constructor Injection Makes Testing Trivial

Because dependencies are passed in explicitly, unit tests never need Spring's ApplicationContext at all — you just pass mocks straight into the constructor.

```
@ExtendWith(MockitoExtension.class)
class NotificationServiceTest {

    @Mock private EmailClient emailClient;
    @Mock private SmsClient smsClient;

    @Test
    void notifyUser_sendsBothEmailAndSms() {
        NotificationService service = new NotificationService(emailClient, smsClient); // plain 'new',

        service.notifyUser("user-1", "Your loan was approved");

        verify(emailClient).send("user-1", "Your loan was approved");
        verify(smsClient).send("user-1", "Your loan was approved");
    }
}
```

What Happens If Spring Can't Find a Bean?

```
// If no PaymentGateway bean is registered at all, Spring throws at startup:
// org.springframework.beans.factory.NoSuchBeanDefinitionException:
//   No qualifying bean of type 'com.bank.PaymentGateway' available

// If TWO beans exist and neither @Primary nor @Qualifier disambiguates:
// org.springframework.beans.factory.NoUniqueBeanDefinitionException:
//   expected single matching bean but found 2: razorPayGateway,stripeGateway
```

Why it matters: DI is what makes a Spring Boot microservice loosely coupled and independently testable — swap RazorPayGateway for StripeGateway, or for a mock in tests, without touching CheckoutService at all.

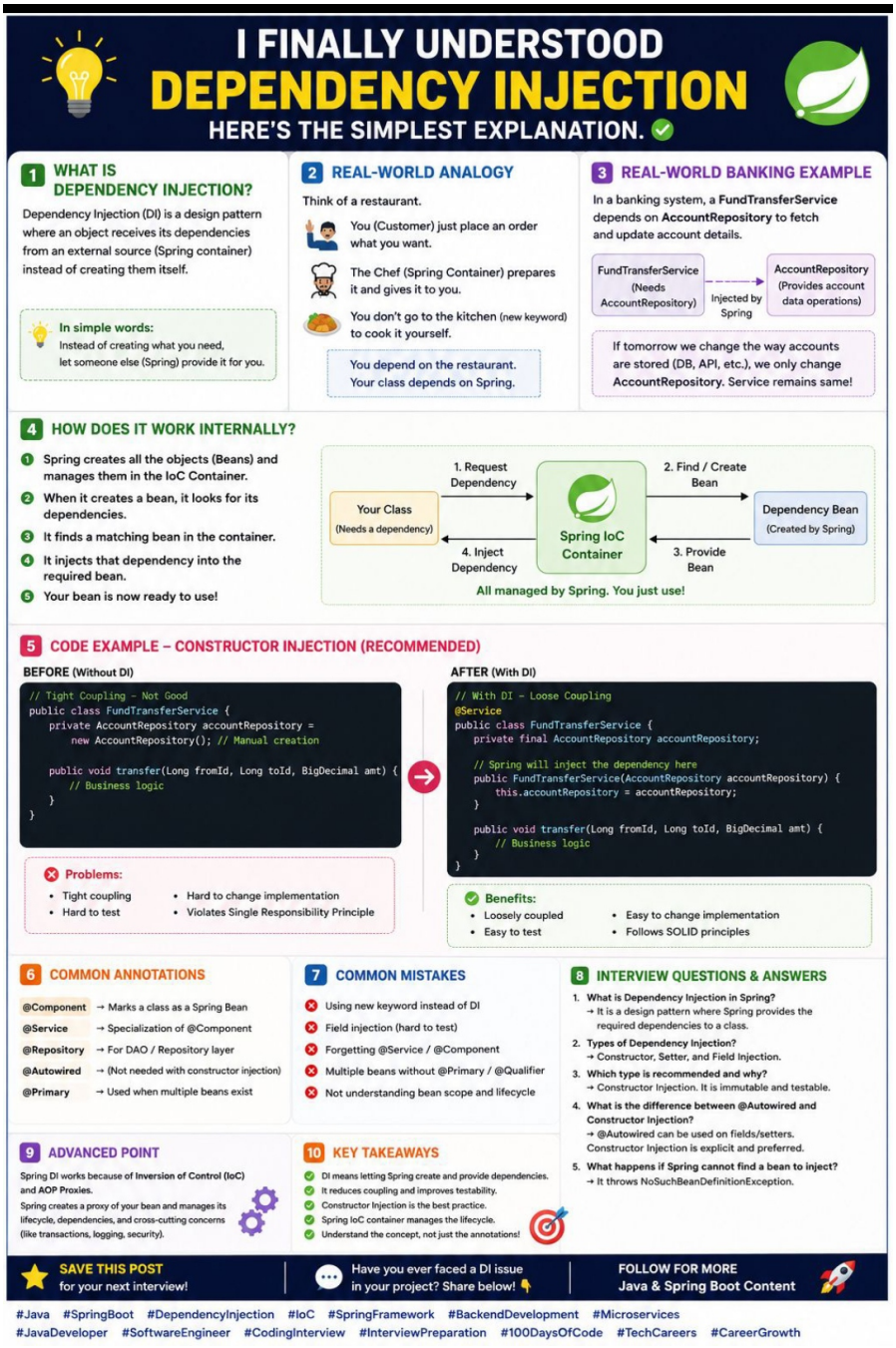


Fig 18. Dependency Injection reference (original infographic).

19. Observability — Extended Hands-On (Spring Boot Microservices)

Definition (Recap)

Observability is the ability to answer *why* a distributed system is behaving a certain way, using three pillars: **metrics** (numeric measurements over time), **logs** (discrete timestamped events), and **traces** (the end-to-end path of a single request across multiple services).

Hands-On: Enabling Actuator + Micrometer + Tracing

```
<!-- pom.xml -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing-bridge-otel</artifactId>
</dependency>
<dependency>
    <groupId>io.opentelemetry</groupId>
    <artifactId>opentelemetry-exporter-otlp</artifactId>
</dependency>

# application.yml
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  tracing:
    sampling:
      probability: 1.0          # trace every request (lower this in production)
  otlp:
    tracing:
      endpoint: http://otel-collector:4318/v1/traces
```

Hands-On: Custom Business Metrics on a Loan Approval Call

```
@Service
public class LoanDecisionService {

    private final Timer decisionTimer;
    private final Counter approvedCounter;
    private final Counter rejectedCounter;

    public LoanDecisionService(MeterRegistry registry) {
        this.decisionTimer = Timer.builder("loan.decision.duration")
                .description("Time taken to reach a loan decision")
                .publishPercentiles(0.5, 0.95, 0.99)
                .register(registry);
        this.approvedCounter = registry.counter("loan.decisions.total", "outcome", "approved");
        this.rejectedCounter = registry.counter("loan.decisions.total", "outcome", "rejected");
    }

    public LoanDecision decide(LoanApplication application) {
        return decisionTimer.record(() -> {
            LoanDecision decision = riskEngine.evaluate(application);
            if (decision.isApproved()) approvedCounter.increment();
            else rejectedCounter.increment();
```

```
            return decision;
        });
    }
}
// Scrape /actuator/prometheus -> loan_decision_duration_seconds, loan_decisions_total{outcome="approv
```

Hands-On: Tracing a Request Across Two Microservices

Micrometer Tracing automatically propagates a trace ID across HTTP calls made with RestClient/WebClient/OpenFeign — no manual header plumbing needed once the tracing starter is on the classpath.

```
// Order Service - calls Payment Service; trace context propagates automatically
@Service
public class OrderService {
    private final RestClient restClient;

    public OrderService(RestClient.Builder builder) {
        this.restClient = builder.baseUrl("http://payment-service").build();
    }

    public PaymentResponse chargeCustomer(Order order) {
        // Micrometer Tracing injects traceparent header automatically via RestClient interceptor
        return restClient.post()
                .uri("/api/gateway/payments")
                .header("Idempotency-Key", order.getIdempotencyKey())
                .body(new PaymentRequest(order.getId(), order.getAmountPaise()))
                .retrieve()
                .body(PaymentResponse.class);
    }
}

// Every log line can include the trace/span id automatically:
// logging.pattern.level=%5p [${spring.application.name},%X{traceId},%X{spanId}]
// Sample log output:
// INFO [order-service,6b19cf9e2b7b4c1c,3a2f9d1e] Order 1042 created, charging payment-service...
```

Hands-On: Minimal docker-compose for a Local Observability Stack

```
# docker-compose.yml
services:
  otel-collector:
    image: otel/opentelemetry-collector:latest
    ports: ["4318:4318"]
  prometheus:
    image: prom/prometheus:latest
    ports: ["9090:9090"]
    volumes: ["./prometheus.yml:/etc/prometheus/prometheus.yml"]
  grafana:
    image: grafana/grafana:latest
    ports: ["3000:3000"]

# prometheus.yml
scrape_configs:
  - job_name: 'order-service'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['order-service:8080']
  - job_name: 'payment-service'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['payment-service:8082']
```

Why it matters: With this stack running, a slow checkout can be diagnosed in minutes: check **loan_decision_duration_seconds** in Grafana for the affected service, then pull the exact trace ID from the logs to see precisely which downstream call (payment gateway, DB query) added the latency.

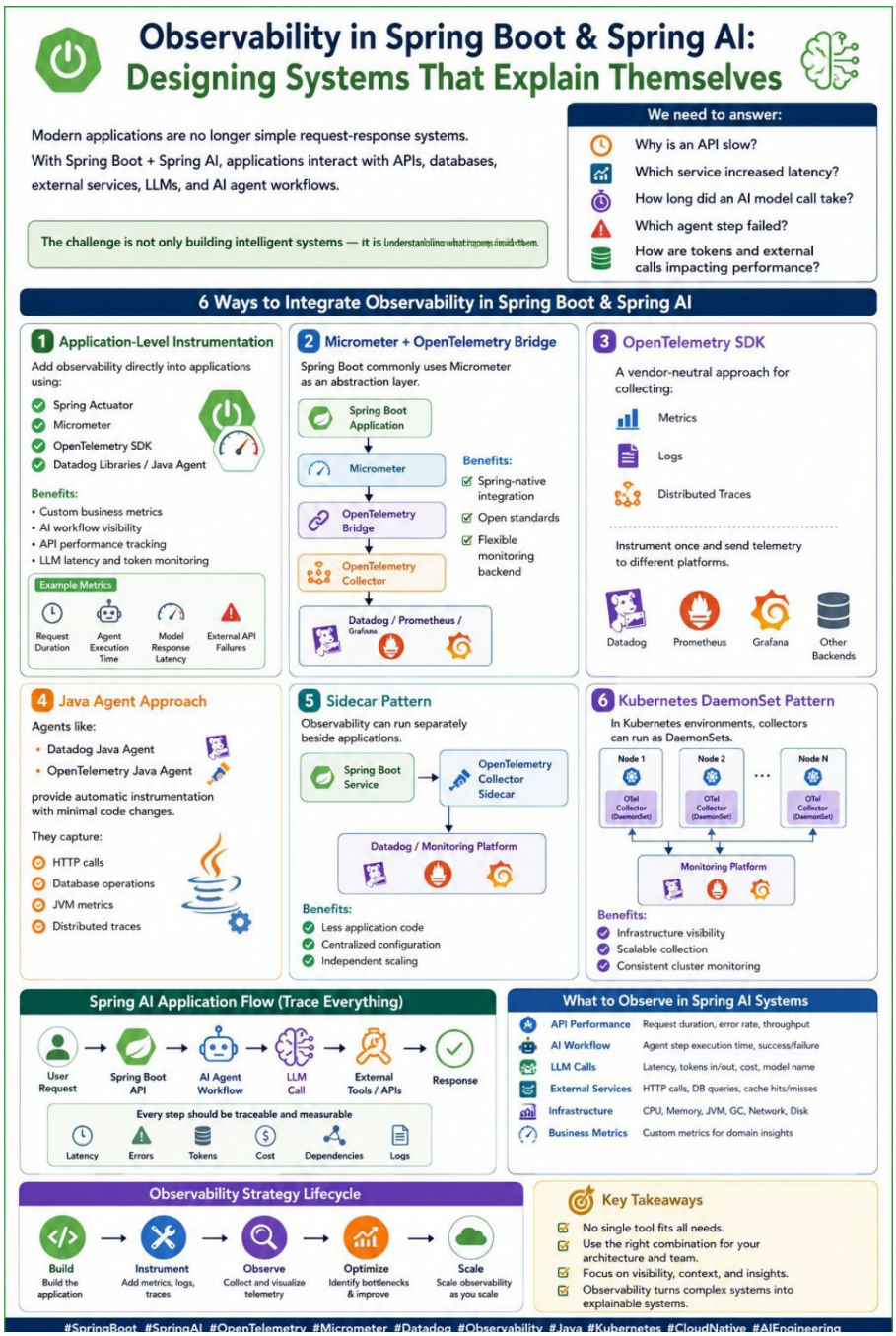


Fig 19. Observability in Spring Boot & Spring AI (original infographic).